Prolog Introduction

From Intuition to Formal Logic

Ibrahim Nasser

Last updated on October 19, 2026

Copyright © 2026 Ibrahim Nasser

Unification—

and backtracking predicates.

Throughout the seasons.

Preface

This document is a hands-on, practical introduction to logic programming with Prolog.

Learning Prolog is a fun and refreshing experience. Instead of telling the computer step-by-step *how* to calculate a result using loops and mutable variables, Prolog encourages you to describe *what* relationships hold true, letting the engine find the answers through unification and backtracking.

The main purpose of this document is to help you build intuition through plenty of practice predicates and concrete examples. We will cover:

- The core mechanics: understanding how the engine searches, matches, and backtracks.
- Thinking recursively: working comfortably with numbers, lists, and trees.
- Classic problems and algorithms: puzzles, search problems, and games.
- A final chapter on theoretical foundations, included purely for completeness if you are curious about the formal First-Order Logic and resolution machinery under the hood.

I hope you enjoy exploring the declarative mindset. Have fun querying!

Contents

Preface ii

1 Introduction 1
- 1.1 Your First Program . . . 1
- 1.2 Prolog Syntax . . . 2

2 Prolog Engine 3
- 2.1 Backchaining . . . 3
- 2.2 Backtracking . . . 4
- 2.3 Functions . . . 5

3 Recursion 7
- 3.1 Recursion in Imperative Programming . . . 7

4 Peano Arithmetics 11

5 Numerical Arithmetic 15
- 5.1 Green Cuts vs. Red Cuts . . . 17
- 5.2 Prolog Tail Recursion . . . 18

6 Lists 20
- 6.1 Higher-Order Predicates . . . 28

7 Fun with Towers of Hanoi 33

8 Binary Trees 36
- 8.1 Binary Search Trees . . . 40

9 Tree Search Algorithms 42

10 MiniMax 54

11 Fun with Propositional Logic 58

12 Theoretical Foundations 61
- 12.1 Logical Systems and First-Order Logic . . . 61
- 12.2 Substitutions and Unification . . . 63
- 12.3 The Resolution Calculus . . . 67
- 12.4 Horn Logic and SLD-Resolution . . . 69

1 Introduction

1.1 Your First Program

In a Prolog program, we write **facts** and **rules** to construct our **knowledge base (KB)**. We can then **query** the KB, and the Prolog engine computes answers via **unification**, **substitution**, and **resolution** (a formal treatment is provided in Section 12).

Example 1.1: Facts

We want to add the facts that Socrates and Leibniz are humans and Socrates is also a Greek:

```
human(socrates).
human(leibniz).
greek(socrates).
```

Observation

This is close to predicate logic syntax:

- human(socrates)
- human(leibniz)
- greek(socrates)

Example 1.2: Rules

We want to add the following rule:

$$\forall X.\text{human}(X) \Rightarrow \text{fallible}(X)$$

We do so in Prolog by writing:

```
fallible(X) :-
    human(X).
```

Example 1.3: Query

Now we can ask: *"Are there any fallible Greeks?"* by querying our knowledge base as follows:

```
?- fallible(X), greek(X).
```

Prolog machinery runs under the hood to return an **answer substitution** [socrates/X]:

```
X = socrates
```

Example 1.4: Ground Query

Alternatively, we can ask a ground question with no variables (*"Is Socrates fallible?"*):

```
?- fallible(socrates).
```

Prolog will return `true`.

Prolog (short for *PROgramming in LOGic*) is a **declarative logic programming language**. Unlike conventional **imperative** languages (such as C, Java, or Python), where a programmer specifies step-by-step instructions detailing *how* the machine should alter state to compute a result, Prolog adopts a **declarative** philosophy: the programmer specifies *what* relationships and truths hold within a problem domain.

As famously formulated by Robert Kowalski (1979):

$$\text{Algorithm} = \text{Logic} + \text{Control}$$

In Prolog, the programmer is primarily responsible for formulating the **Logic** component, formalized as a subset of first-order predicate logic known as *Horn clauses*. The runtime engine supplies the **Control** component through an automated deductive inference mechanism based on **resolution**, **unification**, and depth-first search with **backtracking**.

1.2 Prolog Syntax

Def 1.1. Prolog expresses knowledge about the world via:

- **Constants**: atoms (lowercase strings) or numbers (e.g., `0`, `42`).
- **Variables**: denoted by strings starting with an uppercase letter (e.g., `X`) or an underscore (`_`).
- **Predicates**: relations applied to terms to form truth-valued statements.

Example 1.5

- socrates, leibniz are both constants.
- X is a variable.
- human, greek, and fallible are all predicates.

Def 1.2. A **term** is the fundamental data object in Prolog. A term is either:

- A **constant** (an atom or number),
- A **variable**, or
- A **compound term** (or structure), consisting of a **functor** (function symbol in logic) applied to argument terms, written as $f(t_1, \ldots, t_n)$. Functors act purely as uninterpreted data constructors, not evaluable functions.

Example 1.6

socrates and X are both terms.

Notation

The number of arguments a predicate takes is called its **arity**. In Prolog literature, a predicate is uniquely identified by its name and arity, written as `functor/arity` (e.g., `human/1`, `parent/2`).

Def 1.3. A Prolog **literal** is a predicate symbol applied to terms, $p(t_1, \ldots, t_n)$, expressing an atomic proposition that can be proven true or false.

Example 1.7

human(socrates) is a literal.

Def 1.4. A Prolog program is a sequence of **clauses**. A clause is either:

- A **fact** of the form $l.$ (a literal followed by a period).
- A **rule** of the form h :- $b_1, \ldots, b_n.$ where $n > 0$. The literal h is called the **head** (or **head literal**), and the literals b_i constitute the **body** of the rule.

Example 1.8

human(socrates). is a fact and fallible(X) :- human(X). is a rule.

Def 1.5. A **query** is a list of Prolog literals called **goal literals** (also **subgoals** or simply **goals**). We write a query as `?-`$A_1, \ldots, A_n$`.`, where A_i are goals.

2 Prolog Engine

2.1 Backchaining

Def 2.1. Given a query Q : `?-` $A_1, \ldots, A_n$. and a rule $R : h$:- $b_1, \ldots, b_m$., **backchaining** computes a new query by:

1. finding a substitution that unifies the rule head h with the selected goal A_1, and
2. replacing A_1 in Q with the body literals $b_1, \ldots, b_m$, applying the substitution to all variables.

Note

Backchaining motivates the terms **goals** and **subgoals**:
- The literals in the query are "goals" that must be satisfied.
- Backchaining reduces a goal by replacing it with new "subgoals" from the matching rule body.

Def 2.2. The Prolog interpreter keeps backchaining from the **top** to the **bottom** of the program until the query

- **succeeds**, i.e. contains no more goals, or
- **fails**, i.e. backchaining becomes impossible

At this point, the interpreter issues an **answer**: `true` in the first case and `false` in the second.

Def 2.3. If a query contains variables, then Prolog will return an **answer substitution** as the answer to the query, i.e. the values for all the query variables accumulated during repeated backchaining.

Example 2.1: Backchaining

Recall our knowledge base:

```
(1) human(socrates).
(2) human(leibniz).
(3) greek(socrates).
(4) fallible(X) :- human(X).
```

Suppose we pose the query:

```
?- fallible(Y), greek(Y).
```

The Prolog engine executes repeated **backchaining** steps:

Step 1: Initial Query: Q_0 = ?- $\underline{\text{fallible}(Y)}, \text{greek}(Y)$.
- The engine selects the first goal: $A_1 = \text{fallible}(Y)$.
- It scans the program from top to bottom and matches rule (4): $\text{fallible}(X)$:- $\text{human}(X)$.
- **Unification:** To make the head equal to A_1, it sets $[X \mapsto Y]$.
- **Replacement:** A_1 is replaced by the rule's body $\text{human}(Y)$.

This yields the new query:

$$Q_1 = \text{?- } \underline{\text{human}(Y)}, \text{greek}(Y).$$

Step 2: Resolving with a Fact:
- The engine selects the new first goal: $A_1 = \text{human}(Y)$.
- Scanning from top to bottom, it finds fact (1): human(socrates).
- **Unification:** To make them equal, it binds $[Y \mapsto \text{socrates}]$.
- **Replacement:** Because a fact has an *empty body*, $\text{human}(Y)$ is eliminated completely, and the binding is applied to all remaining goals.

This yields the simplified query:

$$Q_2 = \text{?- } \underline{\text{greek(socrates)}}.$$

Step 3: Final Goal:
- The engine selects $A_1 = \text{greek(socrates)}$.
- It scans and finds fact (3): greek(socrates).
- Unification succeeds trivially (identical terms).
- The goal is eliminated.

This yields the empty query (denoted □ or `true`):

$$Q_3 = \text{?- } \square$$

Since no subgoals remain, the query **succeeds**. Prolog outputs the accumulated variable binding as the **answer substitution**:

```
Y = socrates
```

Step	Current Query	Selected Goal	Matched Clause	Substitution
0	`?- fallible(Y), greek(Y).`	`fallible(Y)`	Rule (4)	$[X \mapsto Y]$
1	`?- human(Y), greek(Y).`	`human(Y)`	Fact (1)	$[Y \mapsto \text{socrates}]$
2	`?- greek(socrates).`	`greek(socrates)`	Fact (3)	$\emptyset$
3	□ (*empty query*)	—	—	**Success:** $Y = \text{socrates}$

2.2 Backtracking

Def 2.4. Prolog Search Procedure: The Prolog interpreter employs **top-down**, **left-to-right** *Depth-First Search*. Concretely, the execution mechanism:

- Resolves subgoals in **left-to-right** order.
- Matches the selected subgoal against the head literals of clauses in **top-to-bottom** order.
- If no clause matches, it fails and **backtracks** to the most recent **choice point** (or **backtrack point**).
- Otherwise, it backchains on the first matching clause and records remaining alternative matches as **choice points**.

We say that a goal G matches a head H iff they can be unified by binding variables to terms.

Note

We can force backtracking to compute more answers by typing `;`

Example 2.2: More Answers

```
human(leibniz).
human(socrates).
human(plato).
greek(socrates).
greek(plato).
fallible(X) :- human(X).
```

If we query `?- fallible(X), greek(X).`, Prolog yields `X = socrates`. Typing `;` prompts Prolog to backtrack, producing the next answer, `X = plato`.

Example 2.3: Backtracking and Choice Points

Consider the following knowledge base:

```
has_wheels(mytricycle, 3).
has_wheels(myrollerblade, 3).
has_wheels(mybmw, 4).
has_motor(mybmw).
car(X) :- has_wheels(X, 3), has_motor(X). % Rule 1
car(X) :- has_wheels(X, 4), has_motor(X). % Rule 2
```

We pose the query to find an entity that is a car:

```
?- car(Y).
```

The execution proceeds through depth-first search with **backtracking**:

Step 1: The goal is `car(Y)`. The engine finds a match with **Rule 1**. Because an untried alternative (**Rule 2**) exists, Prolog creates **Choice Point 1** and substitutes $[X \mapsto Y]$.
- Remaining goals: `has_wheels(Y, 3), has_motor(Y)`.

Step 2: The first subgoal is `has_wheels(Y, 3)`. The engine matches the first fact `has_wheels(mytricycle, 3)`. Because an untried alternative fact exists (`myrollerblade`), Prolog creates **Choice Point 2** and binds $[Y \mapsto$ mytricycle$]$.
- Remaining goal: `has_motor(mytricycle)`.

Step 3: The engine attempts to prove `has_motor(mytricycle)`. Scanning the program from top to bottom reveals no matching facts or rules. The goal **fails**.
- The engine **backtracks** to the most recent choice point (**Choice Point 2**).
- The binding of Y is **undone** (Y becomes unbound again).

Step 4: Prolog tries the next match for `has_wheels(Y, 3)`, matching `has_wheels(myrollerblade, 3)` with $[Y \mapsto$ myrollerblade$]$. Since no further 3-wheel facts exist, Choice Point 2 is now exhausted.
- Remaining goal: `has_motor(myrollerblade)`.

Step 5: `has_motor(myrollerblade)` finds no match and fails.
- Since Choice Point 2 is exhausted, Prolog backtracks further up to **Choice Point 1**.
- All bindings associated with Rule 1 are undone (Y becomes unbound again).

Step 6: Prolog now tries **Rule 2**: `car(X) :- has_wheels(X, 4), has_motor(X).`
- Next subgoal: `has_wheels(Y, 4)`.
- Matches `has_wheels(mybmw, 4)` with binding $[Y \mapsto$ mybmw$]$.
- Next subgoal: `has_motor(mybmw)`. Matches the fact `has_motor(mybmw)`.

No further goals remain ($\square$). The query **succeeds**, and Prolog returns the answer substitution:

```
Y = mybmw
```

2.3 Functions

Excursion: Remember - FOL Signature

Def 2.5. A First-Order Logic Signature is a tuple $\Sigma := \langle \Sigma^f, \Sigma^p \rangle$ where:
- $\Sigma^f := \bigcup_{k \in \mathbb{N}} \Sigma^f_k$ of **function constants** (or **terms**), where members of Σ^f_k denote the k-ary functions on **individuals**.
- $\Sigma^p := \bigcup_{k \in \mathbb{N}} \Sigma^p_k$ of **predicate constants**, where members of Σ^p_k denote k-ary relations among individuals.
- Σ^f_k and Σ^p_k are pairwise disjoint, countable sets of symbols for each $k \in \mathbb{N}$.
- A 0-ary **function constant** refers to a single **individual**; therefore, we call it an **individual constant**.

In the semantics of First-Order Logic:

- A **function constant** $f \in \Sigma^f_k$ is interpreted as a mapping $f^{\mathcal{I}} : D^k \to D$. A term like father(john) semantically translates to an **individual** in the domain (e.g., bob).
- A **predicate constant** $P \in \Sigma^p_k$ is interpreted as a relation $P^{\mathcal{I}} \subseteq D^k$. An atomic formula like loves(john, mary) evaluates strictly to a **truth value** depending on whether the tuple belongs to the relation.

In Prolog, however, the execution engine only evaluates **predicates** to determine truth or falsity. There is no mechanism where an expression evaluates to an individual, a term like `father(john)` will never semantically resolve to the individual `bob`.

To represent a function in Prolog, we must therefore encode it as a **predicate** by adding an extra argument for the output:

```
father(john, bob).
```

Here, `father(john, bob)` evaluates to `true`, while `father(john, alice)` evaluates to `false`. To find who the father of John is, we query:

```
?- father(john, X).
```

where Prolog searches for the individual `X` that makes the predicate true.

3 Recursion

Suppose we have the following knowledge base:

```
parent(terach, abraham).
parent(abraham, isaac).
parent(isaac, jacob).
parent(jacob, joseph).
...
```

We want to define a predicate `ancestor/2` as `ancestor(X,Y)` that is true when `X` is an ancestor of `Y`.

If we think about it, a parent is considered an ancestor, so maybe we can just write:

```
ancestor(X, Y) :- parent(X, Y).
```

But a grandparent is also an ancestor!

```
ancestor(X, Z) :- parent(X, Y), parent(Y, Z).
```

And a great-grandparent!

```
ancestor(X, W) :- parent(X, Y), parent(Y, Z), parent(Z, W).
```

And ...

Problem

With this approach, we can only capture a specific chain of generations (here we only captured 3). What if someone asks about a chain of 10 generations? or 100? or ...? $\rightsquigarrow$ infinite number of rules needed!

Solution

Recursion to the rescue $\rightsquigarrow$ Think Recursively!

We define the ancestor relation recursively. First, we define the **base case**: X is an ancestor of Y if X is the immediate parent of Y:

```
ancestor(X, Y) :-
    parent(X, Y).
```

Now we define the **recursive case**: X is an ancestor of Y if X is a parent of some intermediate person Z, and Z is an ancestor of Y!

```
ancestor(X, Y) :-
    parent(X, Z),
    ancestor(Z, Y).
```

3.1 Recursion in Imperative Programming

Let us review recursion in Python to remember how call stacks work procedurally. Recursion means a function calls itself to solve smaller parts of a problem until it reaches a **base case**, which stops the recursion. Every recursive function needs:

1. A **base case** (when to stop).
2. A **recursive case** (when to keep calling itself).

Example 3.1: Factorial

Mathematically, factorial is recursive:

$$n! = n \times (n-1)! \quad \text{with the base case} \quad 0! = 1$$

We translate that to a Python program:

```
def factorial(n):
    # Base case
    if n == 0:
        return 1
    # Recursive case
    return n * factorial(n - 1)
```

Let's trace `factorial(4)`:

$$\begin{aligned}
\text{factorial}(4) &= 4 \times \text{factorial}(3) \\
&= 4 \times (3 \times \text{factorial}(2)) \\
&= 4 \times (3 \times (2 \times \text{factorial}(1))) \\
&= 4 \times (3 \times (2 \times (1 \times \text{factorial}(0)))) \\
&= 4 \times (3 \times (2 \times (1 \times 1))) = 24
\end{aligned}$$

Observation

Each call pauses until the smaller call finishes. The recursion *"unwinds"* once it hits the base case.

Example 3.2: Fibonacci

The Fibonacci sequence is defined as:

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \text{ for } n \geq 2$$

In Python:

```
def fibonacci(n):
    # Base cases
    if n == 0:
        return 0
    elif n == 1:
        return 1
    # Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Tracing `fibonacci(4)`:

$$\begin{aligned}
\text{fibonacci}(4) &= \text{fibonacci}(3) + \text{fibonacci}(2) \\
&= (\text{fibonacci}(2) + \text{fibonacci}(1)) + (\text{fibonacci}(1) + \text{fibonacci}(0)) \\
&= ((\text{fibonacci}(1) + \text{fibonacci}(0)) + 1) + (1 + 0) \\
&= ((1 + 0) + 1) + (1 + 0) = 3
\end{aligned}$$

Observation

Here you can see multiple recursive branches forming a call tree, because each call spawns two more calls.

Tail Recursion

In **tail recursion**, the recursive call is the *last operation* performed in the function. There is no computation left after the recursive call returns. This allows certain languages (not Python, but others like Scheme or Prolog) to reuse the same stack frame, converting recursion into an iterative process internally. This is known as **tail call optimization (TCO)**.

Accumulator

A clean way to express tail recursion is by defining a **main function** that delegates to a **helper** carrying an **accumulator**. The helper performs the recursion, while the main function provides the initial accumulator value.

Example 3.3: Factorial

Here is a tail-recursive implementation of the `fact` function:

```
def fact(n):
    def fact_helper(n, acc):
        # Base case
        if n == 0:
            return acc
        # Tail-recursive case
        return fact_helper(n - 1, n * acc)
    return fact_helper(n, 1)
```

Tracing `fact(4)`:

$$\begin{aligned}\texttt{fact}(4) &= \texttt{fact_helper}(4, 1)\\ &\to \texttt{fact_helper}(3, 4)\\ &\to \texttt{fact_helper}(2, 12)\\ &\to \texttt{fact_helper}(1, 24)\\ &\to \texttt{fact_helper}(0, 24) = 24\end{aligned}$$

Observation

Each recursive call directly returns another call without additional computation. This makes the recursion tail-recursive, and languages with TCO can execute it in constant stack space.

Example 3.4: Fibonacci

We can apply the same structure to the Fibonacci sequence. The helper carries the previous two Fibonacci numbers as parameters `(a, b)` (think of them as two pointers, `prev` and `current`).

```
def fib(n):
    def fib_helper(n, a, b):
        # Base case
        if n == 0:
            return a
        # Tail-recursive case
        return fib_helper(n - 1, b, a + b)
    return fib_helper(n, 0, 1)
```

Tracing `fib(5)`:

$$\begin{aligned} fib(5) &= fib_helper(5, 0, 1) \\ &\to fib_helper(4, 1, 1) \\ &\to fib_helper(3, 1, 2) \\ &\to fib_helper(2, 2, 3) \\ &\to fib_helper(1, 3, 5) \\ &\to fib_helper(0, 5, 8) = 5 \end{aligned}$$

Observation

Here, `a` represents the previous Fibonacci number and `b` the current one. Each call shifts these accumulators forward until the base case is reached. See figure 1 for illustration.

Tip

Keep this accumulator pattern in mind: in upcoming sections on arithmetic and list processing, we will see how Prolog leverages accumulators to execute recursive predicates in constant stack space via **Last Call Optimization (LCO)**.

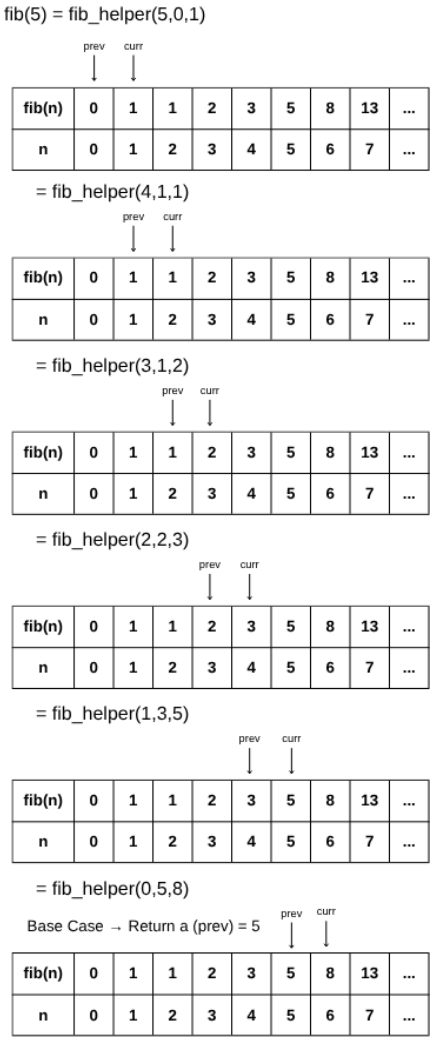


Figure 1: Fib Tail Recursion

4 Peano Arithmetics

Remember

A unary natural number is a term generated by the following grammar rule:

$$u ::= 0 \mid s(u)$$

where 0 denotes zero and $s(u)$ denotes the successor of u. Therefore, the set of unary natural numbers is:

$$\{0, s(0), s(s(0)), s(s(s(0))), \cdots\}$$

Excursion: Peano Axioms

The following set of axioms are called the Peano axioms. They can be used to *reason* about natural numbers:

- **P1.** 0 is a unary natural number.
- **P2.** Every unary natural number has a successor that is a unary natural number and that is different from it.
- **P3.** 0 is not a successor of any unary natural number.
- **P4.** Different unary natural numbers have different successors.
- **P5.** Induction Axiom. Every unary natural number possesses a property P if
  - 0 has the property P, and
  - the successor of every unary natural number that has the property P also possesses property P

Excursion: Unary Operations

Def 4.1. Addition: We define the addition operation inductively:

- Adding 0 to a number does not change it, expressed as: $0 + X = X$.
- Adding the successor of X to Y yields the successor of $X + Y$, expressed as: $s(X) + Y = s(X + Y)$.

Similarly, multiplication and exponentiation are defined as follows:

Def 4.2. Multiplication:

- $n \cdot 0 = 0$, and
- $n \cdot s(m) = n + (n \cdot m)$

Def 4.3. Exponentiation:

- $n^0 = s(0)$, and
- $n^{s(m)} = n \cdot n^m$

In Prolog, we can represent unary natural numbers with a predicate `nat/1` and a functor `s/1`:

```
nat(0).              % 0 is a natural number
nat(s(X)) :- nat(X).  % the successor of a natural number is a natural number
```

This predicate will return true for all the following:

```
nat(0), nat(s(0)), nat(s(s(0))), nat(s(s(s(0)))), ...
```

Question

Can we write unary addition?

Let's think about it. This is a **function** that returns some value given two arguments, so we have to represent that as a 3-ary predicate.

Suppose we want to do that with a predicate `add/3`

```
add(X,Y,Z)        %true if Z is the result of adding X and Y
```

Unary addition is recursive. The base case is:

$$0 + X = X$$

We simply write:

```
add(0,X,X).
```

The recursive case:

$$s(X) + Y = s(X + Y)$$

In Prolog, we declare the third argument (the output) as something of the structure `s(Z)` and in the body we compute that Z as the addition of X and Y.

```
add(s(X), Y, s(Z)) :- add(X, Y, Z).
```

Small test $(3 + 2)$:

```
?- add(s(s(s(0))), s(s(0)), X).
X = s(s(s(s(s(0)))))
```

Question

Can we do subtraction?

Beauty of Prolog

We do not need a separate subtraction predicate! We arranged the arguments in our head to mean addition. We can query the `add/3` predicate itself to have answers for subtractions. Because `add(X, Y, Z)` defines the mathematical relation, subtraction comes for free without writing any new code.

Example 4.1

We want the result of $3 - 1 = 2$. We query our knowledge base to look for the appropriate X such that $X + 1 = 3$:

```
?- add(X, s(0), s(s(s(0)))).
X = s(s(0))
```

Of course, we can define it recursively if we want. Subtraction can be seen as:

- $X - 0 = X$ Base Case
- $s(X) - s(Y) = X - Y$ Recursion

```
% Base case: subtracting 0 from any number X leaves X
sub(X, 0, X).

% Recursive step: (X + 1) - (Y + 1) is the same as X - Y
sub(s(X), s(Y), Z) :-
    sub(X, Y, Z).
```

Small test $(3 - 1 = 2)$:

```
?- sub(s(s(s(0))), s(0), Res).
Res = s(s(0))
```

And we can define it in terms of addition:

```
% sub(A, B, Res) is true if A - B = Res, which means Res + B = A
sub(A, B, Res) :- add(Res, B, A).
```

Similarly, we can represent unary multiplication:

$$X \cdot 0 = 0 \qquad X \cdot s(Y) = X + (X \cdot Y)$$

Sanity Check

$$4 \cdot 3 = 12 \qquad 4 \cdot s(2) = 4 + (4 \cdot 2) = 4 + 8 = 12$$

We write a predicate `mult/3`:

```
% Base case
mult(_, 0, 0).

% Recursive step
mult(X, s(Y), Z) :-
    mult(X, Y, W),
    add(X, W, Z).
```

To see how this works, consider the query computing $2 \times 2 = 4$:

```
?- mult(s(s(0)), s(s(0)), Ans).
```

The execution proceeds by recursively decomposing the second argument down to the base case, and then accumulating the sum on the way back up:

Step 1: The first call `mult(s(s(0)), s(s(0)), Ans)` matches the recursive rule with:

$$X = s(s(0)), \quad Y_1 = s(0)$$

To compute the answer, Prolog creates two subgoals:

(a) Compute the sub-product: `mult(s(s(0)), s(0),` W_1`)`.

(b) Then add: `add(s(s(0)),` W_1`, Ans)`.

Step 2: The second call is the first subgoal `mult(s(s(0)), s(0),` W_1`)`. This matches the recursive rule again with:

$$X = s(s(0)), \quad Y_2 = 0$$

It generates two further subgoals:

(a) Compute: `mult(s(s(0)), 0,` W_2`)`.

(b) Then add: `add(s(s(0)),` W_2`,` W_1`)`.

Step 3: Prolog evaluates `mult(s(s(0)), 0,` W_2`)`. Since the second argument is 0, it directly matches the **base case** `mult(_, 0, 0)`, giving:

$$W_2 = 0$$

Step 4: Unwinding the Additions: Now the engine resumes and resolves the pending additions back up the call stack:

- **Resolving** W_1**:** `add(s(s(0)), 0,` W_1`)` yields:

$$W_1 = s(s(0)) \quad \text{(i.e., } 2 + 0 = 2\text{)}$$

- **Resolving `Ans`:** `add(s(s(0)), s(s(0)), Ans)` yields:

$$\text{Ans} = s(s(s(s(0)))) \quad \text{(i.e., } 2 + 2 = 4\text{)}$$

All goals succeed, and Prolog returns:

```
Ans = s(s(s(s(0))))
```

Note

Division comes for free too

Example 4.2: Division using Multiplication

Prolog finds $4/2 = 2$ without writing a division predicate.

```
?- mult(s(s(0)), X, s(s(s(s(0))))).
X = s(s(0))
```

Similarly, we can represent unary exponentiation:

$$X^0 = s(0) \qquad X^{s(Y)} = X \cdot X^Y$$

Sanity Check

$$3^3 = 3^{s(2)} = 3 \cdot 3^2 = 3 \cdot 9 = 27$$

We write a predicate `uexp/3`:

```
% Base case
uexp(_, 0, s(0)).

% Recursive step
uexp(X, s(Y), Z) :-
    uexp(X, Y, W),
    mult(X, W, Z).
```

Small test ($2^2 = 4$):

```
?- uexp(s(s(0)), s(s(0)), Ans).
Ans = s(s(s(s(0))))
```

5 Numerical Arithmetic

To evaluate arithmetic expressions numerically rather than treating them as symbolic data structures, Prolog provides the built-in infix operator `is`.

Example 5.1

Notice the crucial difference between the standard unification operator `=` and the arithmetic evaluator `is`:

```
?- X = 2 + 3.
X = 2 + 3.

?- X is 2 + 3.
X = 5.
```

In the first query, `X = 2 + 3` performs purely structural unification; Prolog treats `2 + 3` as the symbolic compound term.
In the second query, `X is 2 + 3` instructs the engine to compute the numeric value of the right-hand side expression first, and then unify the result with `X`.

Note

The right-hand side of `is` must be fully instantiated (e.g., X is Y + 1 throws an instantiation error if Y does not already hold a number).

Let's write a predicate `fact(X,Y)` that is true if Y is the factorial of X.

For the base case, we know that $0! = 1$.

```
fact(0,1).
```

For the recursive case, we know that $X! = X \cdot (X-1)!$. We write that as follows:

```
fact(X,Xfact) :-
    X \= 0,                 % \= means not equal
    Y is X-1,
    fact(Y, Yfact),
    Xfact is X * Yfact.
```

Quick Test:

```
?- fact(0,X).
X = 1
;
false
```

```
?- fact(4,X).
X = 24
;
false
```

Problem

Look at what happens at the bottom of the recursion when Prolog evaluates our query:

- Assume the query is just `?- fact(0,X).`
- Prolog matches the first clause.
- But the engine sees that there is a second clause in our KB that also matches the query.

- Therefore, Prolog creates a Choice point on the stack pointing to clause 2, just in case clause 1 doesn't work out.
- This is wasteful: factorial is a deterministic function (there is only one answer), yet Prolog leaves a useless choice point in memory.

Solution

The cut operator `!` to the rescue. To fix this, we tell Prolog: *"Once you match the base case, commit to it! Do not leave a choice point, and do not try any other clauses"*

Example 5.2: Cut Operator

```
fact(0, 1) :- !.
fact(X, Xfact) :-
    X > 0,
    Y is X - 1,
    fact(Y, Yfact),
    Xfact is X * Yfact.
```

Now if we query:

```
?- fact(3, X).
X = 6
```

Keeping the condition $X > 0$ is important

Assume we want to write a predicate `max(X,Y,Z)` where, given two numbers X and Y, Z is their maximum. We can do that as follows:

```
max(X, Y, X) :- X >= Y.
max(X, Y, Y) :- X < Y.
```

Testing.

```
?- max(1,2,X).
X = 2

?- max(2,1,X).
X = 2;
false

?- max(2,2,X).
X = 2;
false
```

Note

In the first query, clause 1 tests $1 \geq 2$ and fails immediately, Prolog moves to clause 2 (the last match) and outputs $X = 2$ (no backtrack points registered).
In the last two queries, clause 1 tests $2 \geq 1$ or $2 \geq 2$ and succeeds, but Prolog sees that clause 2 is still an untried candidate, so it records a choice point.

To solve this, a **cut** is used:

```
max(X, Y, X) :- X >= Y, !.
max(X, Y, Y) :- X < Y.
```

```
?- max(1, 2, X).
X = 2.

?- max(2, 1, X).
X = 2.

?- max(2, 2, X).
X = 2.
```

A programmer might think: *"If $X \geq Y$ was false, then $X < Y$ must be true! Why waste time checking $X < Y$ in clause 2? Let's just omit it!"*:

```
max(X, Y, X) :- X >= Y, !.
max(_, Y, Y).
```

At first glance, everything seems to work just fine:

```
?- max(1, 2, X).
X = 2.

?- max(2, 1, X).
X = 2.

?- max(2, 2, X).
X = 2.
```

But let's try this query:

```
?- max(5, 2, 2).
```

We should get `false` here but actually we get `true`!

- Prolog tests clause 1 and because $2 \neq 5$ it does not unify, it fails before the cut is ever reached!
- Prolog falls through to clause 2 and it unifies perfectly with the query, that's why we get `true`.

5.1 Green Cuts vs. Red Cuts

Cuts are generally classified into two categories based on whether they alter the declarative meaning of the program:

- **Green Cut:** A cut that only improves efficiency. Removing it does not change the set of answers the program produces.
- **Red Cut:** A cut that alters the declarative semantics of the program. Removing it changes the program's output or produces incorrect answers.

Assume we have the following program. Children under 18 get a free ticket (0), everyone else pays 10:

```
ticket(Age, 0) :- Age < 18, !.
ticket(_, 10).
```

Let's run a query for a 10-year-old:

```
?- ticket(10, Price).
Price = 0.
```

Everything looks fine. Only one solution: Price = 0.

Now if we delete the cut:

```
ticket(Age, 0) :- Age < 18.
ticket(_, 10).
```

And run the same query:

```
?- ticket(10, Price).
Price = 0 ;
Price = 10.
```

Observation

With the cut, the set of solutions was $\{\text{Price} = 0\}$. Without the cut, the set of solutions became $\{\text{Price} = 0, \text{Price} = 10\}$

Because deleting the cut changed the answers and produced an incorrect solution, that cut is a **red cut**.

If we have a condition in the second clause, then deleting the cut does not produce an incorrect solution, and the purpose of the cut would be just to prune search branches and avoid storing unnecessary backtrack points:

```
ticket(Age, 0)  :- Age < 18, !.
ticket(Age, 10) :- Age >= 18.

?- ticket(10, Price).
Price = 0.
```

```
ticket(Age, 0)  :- Age < 18.
ticket(Age, 10) :- Age >= 18.

?- ticket(10, Price).
Price = 0;
false
```

As we expressed factorial, we can express Fibonacci, where we have two base cases:

$$\text{fib}(0) = 0 \qquad \text{fib}(1) = 1$$

And the recursion:

$$\text{fib}(X) = \text{fib}(X-1) + \text{fib}(X-2) \quad \forall X.X > 1$$

```
fib(0,0) :- !.
fib(1,1) :- !.
fib(X,Y) :-
    X>1,
    X1 is X-1,
    X2 is X-2,
    fib(X1, X1fib),
    fib(X2, X2fib),
    Y is X1fib + X2fib.
```

5.2 Prolog Tail Recursion

The factorial and Fibonacci predicates we implemented above mimics the head recursion implementation we discussed in Python.

```python
def factorial(n):
    # Base case
    if n == 0:
        return 1
    # Recursive case
    return n * factorial(n - 1)
```

```python
def fibonacci(n):
    # Base cases
    if n == 0:
        return 0
    elif n == 1:
        return 1
    # Recursive case
    return fibonacci(n - 1) + fibonacci(n - 2)
```

It is more efficient in Prolog to write tail-recursive predicates. Let's try and transform the Python tail-recursive implementation into tail-recursive predicates.

Factorial

Prolog

```
fact(X, Y) :-
    fact_helper(X, 1, Y).

fact_helper(0, Acc, Acc) :- !.

fact_helper(X, Acc, Y) :-
    X > 0, !,
    Prev is X - 1,
    Z is Acc * X,
    fact_helper(Prev, Z, Y).
```

Python Equivalent

```python
def fact(n):
    def fact_helper(n, acc):

        if n == 0:
            return acc

        return fact_helper(n - 1, n *
            acc)

    return fact_helper(n, 1)
```

Fibonacci

Prolog

```
fib(X,Y) :-
    fib_helper(X,0,1,Y).

fib_helper(0,Acc,_,Acc) :- !.

fib_helper(X,Acc1,Acc2,Y) :-
    X>0,
    X1 is X - 1,
    Sum is Acc1+Acc2,
    fib_helper(X1, Acc2, Sum, Y).
```

Python Equivalent

```python
def fib(n):
    def fib_helper(n, a, b):

        if n == 0:
            return a

        return fib_helper(n - 1, b, a +
            b)

    return fib_helper(n, 0, 1)
```

Observation

Notice that in both `fact_helper/3` and `fib_helper/4`, the recursive call is the **very last goal** in the clause body. Because no operations remain after it and no choice points are left open, the Prolog engine applies **Last Call Optimization (LCO)**. It reuses the current stack frame, executing the recursion in $O(1)$ stack space (identical to an imperative `while` loop)!

6 Lists

In Prolog, a **list** is an ordered sequence of elements of arbitrary length. The empty list is represented by the atom `[]`, and any non-empty list is inductively constructed from two parts: a **Head** (the first element) and a **Tail** (the list of all remaining elements).

Prolog provides the special bar operator `[H|T]` to destruct and pattern-match lists:

Example 6.1: List Pattern Matching

Consider unifying a list with the `[H|T]` pattern:

```
?- [H|T] = [1, 2, 3, 4].
H = 1,
T = [2, 3, 4].
```

Here, Prolog unifies `H` with the first element, and `T` with the sublist of the remaining elements.
Notice what happens with a single-element list:

```
?- [H|T] = [apple].
H = apple,
T = [].
```

Observation

Two critical rules to remember about `[H|T]`:

- `H` (the Head) is a **single element** (which can be a number, atom, variable, or another list).
- `T` (the Tail) is **always a list** (if there are no elements left, `T` is the empty list `[]`).

Because lists are defined inductively, recursive pattern matching on `[H|T]` allows us to implement a rich variety of powerful operations, ranging from basic searching and numerical aggregations to permutations, sorting, and higher-order list processing.

Tip

Working with lists in Prolog is a great exercise for your brain to get used to recursion. In imperative programming, **loops** are the standard when it comes to lists. In Prolog, you are forced to **think recursively** since recursion is the only *looping* you can get.

Membership

We can write a predicate `is_member(X,L)` that is only true if X is in the list L.

The base case is when X is the head of the list (or the only element in the list):

```
is_member(X,[X|_]).
```

If X is not the original list head, then it is only a member if it is the head of the tail list (the second element), and if not, then the tail of that list (the third element), and so on...

```
is_member(X,[X|_]).
is_member(X,[_|T]):-is_member(X,T).
```

Length

We define a predicate `my_length(L,X)` that is true if X is the length of the list L.

- Base Case: The empty list has a length of 0
- Recursion: The non-empty list has the length of its tail list plus one.

```
my_length([],0).
my_length([_|T],L):-
    my_length(T,TailLength),
    L is TailLength + 1.
```

Elements Summation

We write a predicate `sumlist(L,X)` that is true if X is the sum of the elements in L.

- Base Case: The empty list has no elements, the sum is 0.
- Recursion: The non-empty list has a sum equal to the sum of the tail's elements plus the head.

```
sumlist([],0).
sumlist([H|T],Res):-
    sumlist(T,TailSum),
    Res is TailSum + H.
```

Sum First n Elements

```
sum_first_x(_,0,0):-!.
sum_first_x([H|T],X,Sum):-
    X1 is X - 1,
    sum_first_x(T,X1,TSum),
    Sum is TSum + H.
```

Sum Last n Elements

```
sum_last_x(_,0,0):-!.
sum_last_x(List, X, Sum):-
    sumlist(List,SumAll),
    my_length(List,Length),
    F is Length - X,
    sum_first_x(List, F, ToSubtract),
    Sum is SumAll - ToSubtract.
```

Get Element at Index X

```
get_element(0,[H|_],H):-!.
get_element(X,[_|T],E):-
    X>0,
    X1 is X-1,
    get_element(X1,T,E).
```

Check if Element is the Last One

```
is_last(X,[X]):-!.
is_last(X,[_|T]):-
    is_last(X,T).
```

Maximum Element

We want to write a predicate `max_list(L,X)` that is only true if X is the maximum element in the list L.

We can do that by normal recursion and using the built-in `max/2` predicate. The base case here is a single-element list (because an empty list has no maximum). The recursive case is finding the maximum of the tail (`TailMax`) and then taking the maximum of the head(`H`) and `TailMax`.

```
max_list([X], X):-!.

max_list([H|T], Max) :-
    max_list(T, TailMax),
    Max is max(H, TailMax).
```

If we do not want to use the built-in predicate, we can use Prolog if-else syntax (comparison):

```
max_list([X], X):-!.

max_list([H|T], Max) :-
    max_list(T, TailMax),
    (H >= TailMax -> Max is H ; Max is TailMax).
```

We can also do this using **Tail Recursion**! We seed an accumulator with the first element of the list, then walk down the list updating the accumulator whenever we see a larger number.

```
% Public wrapper: seed accumulator with H
max_list([H|T], Max) :-
    max_acc(T, H, Max).

% Base case: when list is empty, the accumulator is the final answer
max_acc([], Acc, Acc).

% Case 1: Head is greater than Acc -> update accumulator to H
max_acc([H|T], Acc, Max) :-
    H > Acc, !,
    max_acc(T, H, Max).

% Case 2: Head is smaller or equal -> keep current Acc
max_acc([_|T], Acc, Max) :-
    max_acc(T, Acc, Max).
```

Concatenation

We want to write a predicate `concat(L1, L2, Res)` that is true if `Res` is the result of concatenating list `L2` to `L1`.

```
concat([],X,X).
concat([H|T], Y, [H|T1]):-
    concat(T,Y,T1).
```

Note

Prolog offers a built-in predicate `append/3` that does the same thing

Append Element

We write a similar predicate where the second argument is an element (not a list):

```
append_element([], X, [X]).
append_element([H|T],X,[H|T1]):-
    append_element(T,X,T1).
```

Remove First Occurrence

We write a predicate `remove_one(X,L,R)` that is true if R is the list L where the first occurrence of X is removed. The base case here is when X is the head of the list, then we just want R to be the tail of L. The recursive step is when X is not the head H, we keep H and alter the tail to be the list where the first occurrence of X is removed.

```
remove_one(X,[X|T],T):-!.
remove_one(X,[H|T],[H|T2]):-
    H \= X,
    remove_one(X,T,T2).
```

Remove All Occurrences

We want to write a similar predicate, but we want to remove all occurrences of the element X.

```
remove_all(_,[],[]).
remove_all(X,[X|T],Res) :-
    !,
    remove_all(X,T,Res).
remove_all(X,[H|T],[H|T1]):-
    H \= X,
    remove_all(X,T,T1).
```

Remove Duplicates

We want to write a predicate `remove_duplicates(L1, L2)` that is true if `L2` is a copy of `L1` with no duplicates. Here, we do not want to remove all occurrences, neither the first occurrence. For each element that appears more than once, we want to keep the first occurrence, and remove all later occurrences.

Since we want to remove all subsequent occurrences, we can use the predicate `remove_all/3` we defined earlier.

```
remove_duplicates([],[]).
remove_duplicates([H|T], [H|T3]):-
    remove_all(H, T, T2),
    remove_duplicates(T2,T3).
```

Permutations

We want to write a predicate `permutations(L1, L2)` that is only true if `L2` is a possible permuted copy of `L1`.

Example Query:

```
?- permutations([1,2,3],[3,2,1]).
true
```

Additionally, we want to get all possible permutations via Prolog backtracking if we query with a variable in the second argument.

Example Query:

```
?- permutations([1,2,3],X).

X = [1, 2, 3];
X = [2, 1, 3];
X = [2, 3, 1];
X = [1, 3, 2];
X = [3, 1, 2];
X = [3, 2, 1].
```

Remember our `remove_one` predicate:

```
?- remove_one(1,[1,2,3],X).
X = [2, 3]
```

Question

What if we used it the other way around? We want to provide the element to be removed, the resulted list, and query for the possible original list.

```
?- remove_one(1,X,[2, 3]).
X = [1, 2, 3]
```

Problem

That is great, but we want backtracking to give us all possible original lists.

Idea

Remove the cut from the base case.

```
remove_one(X,[X|T],T).          %cut removed
remove_one(X,[H|T],[H|T2]):-
    H \= X,
    remove_one(X,T,T2).
```

Let's try that again:

```
?- remove_one(1,X,[2, 3]).

X = [1, 2, 3];
X = [2, 1, 3];
X = [2, 3, 1].
```

That exactly is what we hoped for! We can use that idea in our permutation predicate!

We can write the base case as permuting an empty list gives us an empty list as well. But what about the recursive step?

Idea

A permutation `Perm` of a non-empty list `[H|T]` is actually the result if we do that `remove_one` trick to the permuted tail where the element is the head! i.e. it is `remove_one(H,Perm,Tperm)` where we get `Tperm` from `permutations(T,Tperm)`

Let's write this!

```
permutations([],[]).
permutations([H|T],Perm):-
    permutations(T,Tperm),
    remove_one(H,Perm,Tperm).
```

Let's test it:

```
permutations([1,2,3],[3,2,1]).
true
```

```
?- permutations([1,2,3],X).

X = [1, 2, 3];
X = [2, 1, 3];
X = [2, 3, 1];
X = [1, 3, 2];
X = [3, 1, 2];
X = [3, 2, 1].
```

Sort

We want to write a predicate `sort_list(L1, L2)` that is only true if `L2` is the sorted (ascending) version of `L1` (duplicates are allowed).

```
?- sort_list([4,2,6,1,9,2,1,3],X).
X = [1, 1, 2, 2, 3, 4, 6, 9]
```

Idea

If you think about this, the sorted version is just one of the permutations of the list that is sorted!

Idea

Let's first write a predicate that checks if a list is sorted!

We write a predicate `sorted/1`:

- The empty list is considered sorted (trivially)
- The list with one element is sorted as well (also trivially)
- Recursion applies for lists with two or more elements. It will be sorted if the first element is less than or equal the second element and the tail itself (second element included) is sorted!

```
sorted([]).
sorted([_]).
sorted([H1,H2|T]):-
    H1=<H2,
    sorted([H2|T]).
```

```
?- sorted([1,1,2,3,4,4,5]).
true
```

Now for the `sort_list` predicate, the base case is when we have an empty list, we just return an empty list. For the non-empty list, all we have to do is to look for a permutation of the list where that permutation is sorted! Once we find that, we cut!

```
sort_list([],[]).
sort_list(X,Y):-
    permutations(X,Y),
    sorted(Y),!.
```

```
?- sort_list([6,2,4,8,1,3,1],X).
X = [1, 1, 2, 3, 4, 6, 8]
```

Fun Fact

Defining sort as that is the classic **Permutation Sort** (or Slowsort). It runs in $\mathcal{O}(n \cdot n!)$ time which is the purest declarative (yet computationally terrible) sorting algorithm in computer science.

Question

What if we also would like to remove the duplicates?

Answer

Well, we just use our `remove_duplicates`!

```
sort_list([],[]).
sort_list(X,Y):-
    permutations(X,Y1),
    sorted(Y1),!,
    remove_duplicates(Y1,Y).
```

```
?- sort_list([9,2,1,8,1,6,19,2,6],X).
X = [1, 2, 6, 8, 9, 19]
```

Note

Prolog provides a built-in `sort/2` predicate that does exactly that.

Zip

We want to write a predicate `zip(L1,L2,L3)` where `L3` is the result of zipping two lists `L1` and `L2`.

Example Usage:

```
?- zip([1,2,3,4], [5,6], X).
X = [(1,5), (2,6)]

?- zip([1,2], [3,4,5], X).
X = [(1,3), (2,4)]
```

Note

The resulted list has as many elements as the smaller input list.

To make sure we stop the recursion when the smaller list is empty, we make two base cases:

- Zipping the empty list with any `L2` gives us an empty list (`L1` is smaller)
- Zipping any `L1` with the empty list gives us an empty list (`L2` is smaller)

The recursion step for non-empty lists starts by zipping the two heads together as the first element and the tail is the result of zipping the two tails!

```
zip([],_,[]) :- !.
zip(_,[],[]) :- !.
zip([H1|T1], [H2|T2], [(H1, H2)|T3]) :-
    zip(T1, T2, T3).
```

Flatten

We want to write a predicate `flatten_list(L1, L2)` that is true when `L2` is flattened list of a list of lists `L1`.

Example Usage:

```
?- flatten_list([[1,2],3,[4,[5,[6,7,[8]]]]],X).
X = [1, 2, 3, 4, 5, 6, 7, 8]
```

The base case would be flattening an empty list would just give an empty list back.

For recursion, we have two cases:

- The head itself is a list
- The head is just an element

Note

To check if something is a list, we can use the Prolog built-in `is_list/1`.

If the head is a list, we need to flatten the head first, flatten the tail, and then append those flattened versions together.

If the head is a single element, we just output a new list with the same head where the tail is flattened!

Putting things together:

```
flatten_list([], []).

flatten_list([H|T], Res) :-
    is_list(H), !,
    flatten_list(H, Hflat),
    flatten_list(T, Tflat),
    append(Hflat, Tflat, Res).

flatten_list([H|T], [H|T1]) :-
    flatten_list(T, T1).
```

```
?- flatten_list([[[1,2,3],4,5],[[6]]],X).
X = [1, 2, 3, 4, 5, 6]
```

Note

Prolog provides a built-in predicate `flatten/2` that does that.

Reverse Order

We want to write a predicate `reverse_order(L1, L2)` that is true if `L2` contains the elements of `L1` in reverse order.

```
reverse_order([],[]).
reverse_order([H|T],Res):-
    reverse_order(T,TailRev),
    append(TailRev,[H],Res).
```

Note

Prolog provides a built-in predicate `reverse/2` that does the same thing.

Remove Elements

We want to write a predicate `remove_elements(L1, L2, L3)` that is only true if `L3` is a copy of `L1` where all occurrences of elements in `L2` (and `L1`) are removed.

Example Usage:

```
?- remove_elements([1,2,3,4,3,1,4], [1,3,6],X).
X = [2, 4, 4]
```

Obviously here we need to check membership, so we will use our `is_member/2`:

```
is_member(X, [X|_]) :- !.              % Added cut to prevent useless backtrack point
is_member(X, [_|T]) :- is_member(X, T).
```

For the base case, if we want to remove elements from the empty list, we just give the empty list (no intersection).

For non-empty lists, we have two recursive cases:

- The head of `L1` is a member of `L2` $\rightsquigarrow$ We discard it and call the predicate on the tail.
- The head is not a member $\rightsquigarrow$ We keep it and call the predicate on the tail.

```
remove_elements([],_, []).

remove_elements([H|T], List, Res) :-
    is_member(H, List), !,
    remove_elements(T, List, Res).

remove_elements([H|T], List, [H|T1]) :-
    remove_elements(T, List, T1).
```

```
?- remove_elements([1,2,3,4,4,3,2,1], [1,3],X).
X = [2, 4, 4, 2]
```

Hide

`hide/3` is similar to `remove_elements/3`, but instead of removing the elements, we replace them with the constant `hidden`:

```
?- hide([1,2,1,4,3,9], [1,1,9,4], X).
X = [hidden, 2, hidden, hidden, 3, hidden]
```

The implementation is straightforward:

```
hide([],_, []).

hide([H|T], Elements, [hidden|T1]) :-
    is_member(H, Elements), !,
    hide(T, Elements, T1).

hide([H|T], Elements, [H|T1]) :-
    hide(T, Elements, T1).
```

6.1 Higher-Order Predicates

In logic programming, a **higher-order predicate** (or meta-predicate) is a predicate that takes *other predicates* or goals as arguments, enabling us to treat code as data and write generic, reusable abstractions.

To dynamically invoke a predicate passed as a variable argument, Prolog provides the built-in `call/N` family:

- `call(Goal)` executes `Goal` directly.
- `call(Pred, Arg1, ..., ArgN)` appends the arguments to `Pred` and invokes the resulting goal. For example, `call(square, 3, Y)` dynamically constructs and executes the query `square(3, Y)`.

Map

We want to write a predicate `map(P, L1, L2)` that is only true if `L2` contains the elements of `L1` where predicate `P` is applied to each of them.

For example, assume we have a predicate that squares a number X:

```
square(X,Y) :- Y is X*X.
```

We want the following behaviour of our `map/3` if we pass `square` as the argument `P`:

```
?- map(square, [1,2,3], X).
X = [1, 4, 9]
```

For the base case, if `L1` is empty, then `L2` is empty as well (regardless of what predicate we passed).

If we have a non-empty list `[H|T]`, we want `L2` to have `H2` where `H2` is the result of **calling** `square` on `H1` (here where we use the built-in `call/N`). The tail of `L2` is the result of recursively calling `map` on the tail `T` passing the same `square`.

```
map(_,[],[]).

map(P,[H|T],[H1|T1]) :-
    call(P,H,H1),
    map(P,T,T1).
```

```
?- map(square, [4,5], X).
X = [16, 25]
```

Remove If

We want to write a predicate `remove_if(P, L1, L2)` that is true if `L2` is a copy of `L1` with all elements for which the predicate `P` holds removed. Think of this as filtering: for example, filtering out all even elements, all odd elements, and so on.

Assume we have the following `odd/1` and `even/1` predicates:

```
even(X) :- 0 is X mod 2.

odd(X) :- \+ even(X).
```

Note

In Prolog, `\+` represents **Negation as Failure (NAF)**, read as *"not provable"*. The goal `\+ Goal` succeeds if `Goal` fails to be proven from the knowledge base, and fails if `Goal` succeeds. Thus, `odd(X)` simply succeeds whenever `even(X)` fails. This is similar to non-derivability ($\nvdash$) in proof calculi:

$$\backslash\!+A \Leftrightarrow \text{KB} \nvdash A$$

We expect the following behavior from our predicate:

```
?- remove_if(even,[1,2,3,4,5,6],X).
X = [1, 3, 5]
```

If we provide an empty list we just output an empty list (base case). If we have a non-empty list, we have two recursive cases:

- The passed predicate holds for the head $\rightsquigarrow$ Discard the head and recurse on the tail.
- Otherwise, we keep the head and recurse on the tail.

```
remove_if(_, [], []).

remove_if(Pred, [H|T], Res) :-
    call(Pred, H), !,
    remove_if(Pred, T, Res).

remove_if(Pred, [H|T], [H|T1]) :-
    remove_if(Pred, T, T1).
```

```
?- remove_if(odd,[1,2,3,4,5,6],X).
X = [2, 4, 6]
```

Excursion: Alternative Solution

Using if-else syntax instead of two recursive cases:

```
remove_if(_, [], []).

remove_if(Pred, [H|T], Res) :-
    (   call(Pred, H)
    ->  remove_if(Pred, T, Res)
    ;   Res = [H|T1],
        remove_if(Pred, T, T1)
    ).
```

Tip

Speaking of if-else, instead of scattering `(-> ;)` everywhere in your code, you can define a reusable higher-order control block using `call/1`

Example 6.2: Reusable Control Structure

```
if_then_else(Condition, Then, Else) :-
    (   call(Condition)
    ->  call(Then)
    ;   call(Else)
    ).
```

Therefore, the alternative solution can be written as:

```
remove_if(_, [], []).
remove_if(Pred, [H|T], Res) :-
    if_then_else(
        call(Pred, H),
        remove_if(Pred, T, Res),
        (Res = [H|T1], remove_if(Pred, T, T1))
    ).
```

Pack

We want to write a predicate `pack(L1, L2)` that packs similar consecutive elements in `L1` in a list inside `L2`. For example:

```
?- pack([a,a,a,b,b,a,c,c,c], X).
X = [[a, a, a], [b, b], [a], [c, c, c]]
```

We have two base cases:

- Packing an empty list ⇝ empty list
- Packing a list with one element ⇝ a list containing a single sublist, `[[X]]`.

For lists with two or more elements, we compare the first two adjacent items:

- **The first two elements are the same** ⇝ they belong to the same group. We attach the first element to the front of the ongoing group formed by packing the rest of the list.
- **The first two elements are different** ⇝ the current run has ended. We seal the first element into its own completed group, and begin packing a new group starting from the second element.

Putting it all together, we obtain:

```
pack([], []) :- !.
pack([X], [[X]]) :- !.
pack([X, X|T], [[X|Xs]|TPacked]) :- !,
    pack([X|T], [Xs|TPacked]).
pack([X, Y|T], [[X]|TPacked]) :-
    pack([Y|T], TPacked).
```

Encode

We want to write a predicate `encode(L1, L2)` where consecutive duplicates of elements in `L1` are compressed into pairs of the form `[Count, Element]` in `L2` (known as **run-length encoding**):

```
?- encode([a,a,a,b,b,c,a,a,d,e,e,e,e], X).
X = [[3, a], [2, b], [1, c], [2, a], [1, d], [4, e]]
```

Idea

We can use our `pack`!

If we call `pack`, we already get something useful:

```
?- pack([a,a,a,b,b,c,a,a,d,e,e,e,e], X).
X = [[a, a, a], [b, b], [c], [a, a], [d], [e, e, e, e]]
```

Question

What can we do with this ?

Idea

We can create a helper predicate that takes a list of consecutive duplicates and gives us a list with the count and the element.

```
to_rle([H|T], [Count, H]) :-
    length([H|T], Count).
```

```
?- to_rle([a,a,a], X).
X = [3, a]
```

Question

Okay ... What can we do with that?

Idea

Use our `map/3` to apply that helper to each element of the resulted list from `pack`!

```
encode(List, Encoded) :-
    pack(List, Packed),
    map(to_rle, Packed, Encoded).
```

That works just fine!

```
?- encode([a,a,b,c,c,c], X).
X = [[2, a], [1, b], [3, c]]
```

Excursion: Different Approach

Here is a different implementation for the `encode/2` predicate.

```
% Public wrapper
encode([], []) :- !.
encode([X|Xs], Encoded) :-
    encode_run(Xs, X, 1, Encoded).

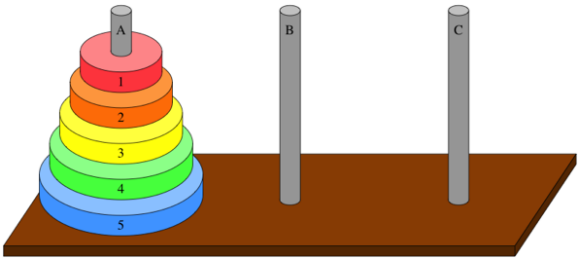
% Base case
encode_run([], Prev, Count, [[Count, Prev]]) :- !.

% Case 1: Same element -> increment count and continue
encode_run([X|Xs], X, Count, Encoded) :- !,
    NextCount is Count + 1,
    encode_run(Xs, X, NextCount, Encoded).

% Case 2: New element -> close current run [Count, Prev], start new run at 1
encode_run([X|Xs], Prev, Count, [[Count, Prev]|Rest]) :-
    encode_run(Xs, X, 1, Rest).
```

7 Fun with Towers of Hanoi

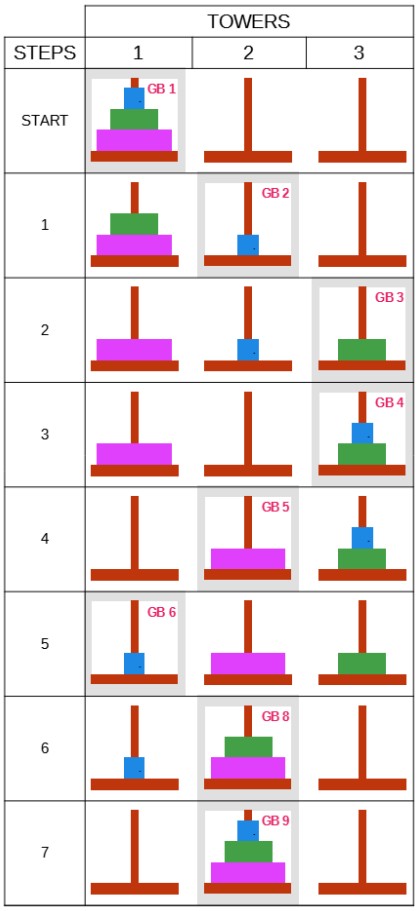
The Towers of Hanoi is a puzzle that consists of three pegs (A, B, and C) and a number of disks of different sizes, which can slide onto any peg. The puzzle starts with the disks arranged in a stack in ascending order of size on one peg, with the smallest disk at the top, forming a conical shape.



The objective of the puzzle is to move all disks from peg A to peg B using the auxiliary peg C, while obeying the following rules:

1. Only one disk may be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty peg.
3. No larger disk may be placed on top of a smaller disk.

Here is an illustration of the steps needed to solve the puzzle for three disks:



We want to write a predicate `move(N, Source, Target, Aux)` that prints the exact sequence of individual moves required to solve the puzzle for N disks.

Think Recursively

If we think about the problem recursively, when we have $N > 1$ disks, we want to move the top $N - 1$ disks to the auxiliary peg, then move the one biggest disk to the target peg, then move the $N - 1$ disks from the auxiliary to the target peg. The base case is trivial: moving one disk requires just a single direct move from source to target!

Prolog Printing

- We will use the `write(X)` predicate that prints the value of X (X can be simple text or any type of argument) to the screen.
- We will use `nl` that prints a new line.

Let's write the base case (one disk):

```
move(1, A, B, _) :- !,
    write("Move top disk from "),
    write(A), write(" to "), write(B), nl.
```

This is what we will get when we call it:

```
?- move(1, left,center,right).
Move top disk from left to center
true
```

The recursion:

```
move(N,A,B,C) :-
    N>1,
    N1 is N-1,
    move(N1,A,C,B),
    move(1,A,B,_),
    move(N1,C,B,A).
```

Let's test this:

```
?- move(3, left, center,right).
Move top disk from left to center
Move top disk from left to right
Move top disk from center to right
Move top disk from left to center
Move top disk from right to left
Move top disk from right to center
Move top disk from left to center
true
```

Excursion: Towers of Hanoi Time Complexity

Let $T(N)$ be the number of moves needed to move N disks.
If there is only one disk, you move it once (base case):

$$T(1) = 1$$

For the recursive case, we want to move $N > 1$ disks from peg A to B using C:

1. Move $N-1$ disks from A to C (*that takes* $T(N-1)$ *moves*)
2. Move the largest disk from A to B (*one move*)
3. Move the $N-1$ disks from C to B (*another* $T(N-1)$ *moves*)

Therefore:

$$T(N) = T(N-1) + 1 + T(N-1) = 2T(N-1) + 1$$

Computing $T(N-1)$ the same way gives us

$$T(N-1) = T(N-2) + 1 + T(N-2) = 2T(N-2) + 1$$

Substitute:

$$2T(N-1) + 1 = 2(2T(N-2) + 1) + 1$$

Expand:

$$2(2T(N-2) + 1) + 1 = (4T(N-2) + 2) + 1 = 4T(N-2) + 3$$

Computing $T(N-2)$ the same way gives us

$$T(N-2) = T(N-3) + 1 + T(N-3) = 2T(N-3) + 1$$

Substitute:

$$4T(N-2) + 3 = 4(2T(N-3) + 1) + 3$$

Expand:

$$(8T(N-3) + 4) + 3 = 8T(N-3) + 7$$

And so on $\cdots$

Observation

For a recursive step k, we get the following:

- $k = 1 \rightarrow T(N) = 2T(N-1) + 1$
- $k = 2 \rightarrow T(N) = 4T(N-2) + 3$
- $k = 3 \rightarrow T(N) = 8T(N-3) + 7$

This can be generalized to:

$$T(N) = 2^k T(N-k) + (2^k - 1)$$

We stop the recursion when we reach the base case $T(1)$ which occurs when $N - k = 1$ (that is, when $k = N-1$).
For $k = N-1$, we have

$$\begin{aligned} T(N) &= 2^{N-1} T(N-(N-1)) + (2^{N-1} - 1) \\ &= 2^{N-1} T(1) + (2^{N-1} - 1) \\ &= 2^{N-1} + (2^{N-1} - 1) \\ &= 2 \cdot 2^{N-1} - 1 \\ &= 2^N - 1 \end{aligned}$$

The complexity is $\mathcal{O}(2^N - 1) = \mathcal{O}(2^N)$, i.e., **exponential**. For our 3-disk example, $T(3) = 2^3 - 1 = 7$ moves, exactly matching the 7 lines of output above.

8 Binary Trees

In Prolog, we do not need special pointers or object classes to represent hierarchical data structures. Instead, we represent a **binary tree** inductively using compound terms:

- **The empty tree** is represented by the atom `nil`.
- **A non-empty node** is represented by the 3-ary functor `tree(Value, Left, Right)`, where `Value` is the stored item, and `Left` and `Right` are themselves binary subtrees.

For example, a leaf node containing 1 is written as `tree(1, nil, nil)`, while a tree with root 1 and children 2 and 3 is represented as:

`tree(1, tree(2, nil, nil), tree(3, nil, nil))`

Structural Validation: `is_b_tree/1`

Before performing operations on trees, we want to write a recognizer predicate `is_b_tree(T)` that succeeds if and only if `T` is a structurally well-formed binary tree. The inductive definition is straightforward:

- **Base Case:** The empty tree `nil` is a valid binary tree.
- **Recursive Step:** A compound term `tree(_, L, R)` is a valid binary tree if both its left subtree `L` and right subtree `R` are valid binary trees.

```
is_b_tree(nil).
is_b_tree(tree(_, L, R)) :-
    is_b_tree(L),
    is_b_tree(R).
```

We can verify our representation:

```
?- is_b_tree(tree(1, tree(2, nil, nil), tree(3, nil, nil))).
true.
?- is_b_tree(tree(1, invalid_child, nil)).
false.
```

Count Nodes

We want to count the nodes in a binary tree with the predicate `bt_nodes_count(Tree, Count)`. For the base case, the empty tree has zero nodes. For non-empty trees, the number of nodes is the number of nodes in the left sub-tree added to the number of nodes in the right sub-tree added to one (the root of the tree).

```
bt_nodes_count(nil,0):-!.
bt_nodes_count(tree(_, L, R), Res):-
    bt_nodes_count(L,LeftNodes),
    bt_nodes_count(R,RightNodes),
    Res is LeftNodes + RightNodes + 1.
```

```
?- bt_nodes_count(tree(5,tree(3,nil,nil),tree(7,nil,nil)), X).
X = 3
```

Count Leaves

`bt_leaves_count/2` should count only the leaves of a BT. We have two base cases:

- Empty tree $\rightsquigarrow 0$
- A leaf (`tree(_,nil,nil)`) $\rightsquigarrow 1$

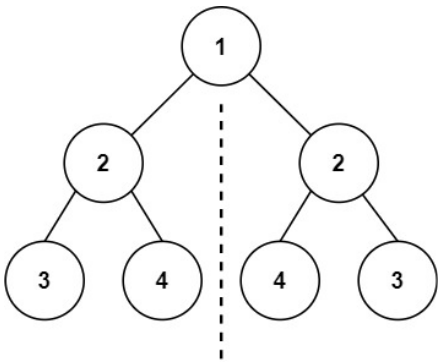
Otherwise, the sum of the leaves count from the right and left subtrees!

```
bt_leaves_count(nil,0):-!.
bt_leaves_count(tree(_,nil,nil),1):-!.
bt_leaves_count(tree(_, L, R), Res):-
    bt_leaves_count(L,LeftLeaves),
    bt_leaves_count(R,RightLeaves),
    Res is LeftLeaves + RightLeaves.
```

```
?- bt_leaves_count(tree(5,tree(3,nil,nil),tree(7,nil,nil)), X).
X = 2
```

Symmetric

We want to write a predicate `symmetric/1` that is only true if the binary tree is symmetric. A symmetric BT would look exactly the same if it was mirrored, i.e. flipped horizontally (the subtrees **mirror** each other).



Idea

Let's write a helper predicate `mirror/2` that checks if two BTs mirror each other!

The empty tree mirrors itself. For two non-empty trees, they mirror each other if their subtrees do!

```
mirror(nil,nil):-!.
mirror(tree(N,L1,R1), tree(N,L2,R2)):-
    mirror(L1,R2),
    mirror(R1,L2).
```

Let's test that on the subtrees of the image above:

```
?- mirror(tree(2, tree(3, nil,nil), tree(4, nil,nil)),
       tree(2, tree(4, nil,nil), tree(3, nil,nil))).
true
```

Idea

Checking symmetry becomes easier now. The empty tree is symmetric. The non-empty tree is symmetric if its left subtree mirrors its right subtree!

```
symmetric(nil):-!.
symmetric(tree(_,L,R)):-
    mirror(L,R).
```

```
?- symmetric(tree(1,
                  tree(2, tree(3, nil,nil), tree(4, nil,nil)),
                  tree(2, tree(4, nil,nil), tree(3, nil,nil))
              )).
true
```

DFS Traversal

We want to traverse a binary tree using in-order Depth-First Search. This will be straight-forward, we just traverse the left sub-tree, print the root label, and traverse the right sub-tree!

```
bt_dfs_trav(nil):-!.
bt_dfs_trav(tree(N,L,R)):-
    bt_dfs_trav(L),
    write(N), write(" "),
    bt_dfs_trav(R).
```

```
?- bt_dfs_trav(tree(1,
                  tree(2, tree(3, nil,nil), tree(4, nil,nil)),
                  tree(2, tree(4, nil,nil), tree(3, nil,nil))
              )).

              3 2 4 1 4 2 3
true
```

BFS Traversal

Breadth-First Search Traversal is a tricky one. We are hoping to print the nodes labels level by level:

```
1 2 2 3 4 4 3
```

Excursion: Remember BFS

The distinguishing feature of BFS is the use of a **Queue** operated on a **FIFO (First-In, First-Out)** basis. Adding newly discovered children to the *back* of the queue guarantees that older sibling nodes waiting at the *front* are processed first.

General Steps:

1. **Initialization:** Create an empty queue Q and enqueue the root node:

$$Q \leftarrow [\text{Root}]$$

2. **Loop until queue is empty:** While Q is not empty:
   (a) **Dequeue:** Remove the node u from the **front** of Q.
   (b) **Visit:** Process node u (e.g., print its value or check a search goal).
   (c) **Enqueue Children:** Add all unvisited children of u to the **back** of Q.
3. **Termination:** When the queue becomes empty, all reachable nodes have been visited.

Problem

In languages like Python or Java, BFS is straightforward because we can instantiate a mutable `Queue` object and run a `while` loop. But in Prolog:

- We have **no `while` loops**.
- We have **no mutable queue data structures**.
- Prolog's execution model is naturally **Depth-First** (because its execution stack is a LIFO stack!).

Question

How do we force a depth-first, immutable language to traverse a tree breadth-first?

List-as-Queue Trick

The answer lies in passing an explicit **Queue as a list** across recursive calls:

1. **Popping from the front:** Is just standard list pattern-matching: `[CurrentNode | Rest]`.
2. **Pushing to the back:** Is list concatenation using `append/3`:

   ```
   append(Rest, [L, R], NewQueue)
   ```

   By placing children at the *end* of the list, they patiently wait behind all nodes on the current level.
3. **The While Loop:** Is replaced by a tail-recursive predicate `bfs_loop(Queue)`.

Question

what if a child is `nil`?

Answer

Instead of writing complicated checks before adding children, we simply dump `[L, R]` onto the queue unconditionally. When a `nil` eventually bubbles up to the front, a single one-line clause quietly discards it!

This entire algorithm boils down to just two clean predicates:

```
% 1. Entry Point: Initialize the queue with the root tree
bt_bfs_trav(nil) :- !.
bt_bfs_trav(Tree) :-
    bfs_loop([Tree]).

% 2. The Worker Loop
% Base case: Queue is empty -> traversal finished!
bfs_loop([]) :- !.

% Skip empty branches: When 'nil' reaches the front, discard it!
bfs_loop([nil|Rest]) :- !,
    bfs_loop(Rest).

% Process node: Print value, and enqueue children [L, R] to the back!
bfs_loop([tree(N, L, R)|Rest]) :-
    write(N), write(" "),
    append(Rest, [L, R], NewQueue),
    bfs_loop(NewQueue).
```

Example 8.1

Testing on a balanced binary tree:

```
?- Tree = tree(1,
               tree(2, tree(4, nil, nil), tree(5, nil, nil)),
               tree(3, tree(6, nil, nil), tree(7, nil, nil))),
   bt_bfs_trav(Tree).
1 2 3 4 5 6 7
true.
```

8.1 Binary Search Trees

While a general binary tree only specifies a hierarchical structure, a **Binary Search Tree (BST)** enforces a strict **ordering invariant**:

For every node with value V, every element in its **left subtree** must be strictly smaller than V, and every element in its **right subtree** must be strictly greater than V.

This ordering property allows efficient $\mathcal{O}(\log N)$ search, insertion, and retrieval.

Node Insertion: `bst_ins/3`

We want to write a predicate `bst_ins(Element, OldTree, NewTree)` that inserts an `Element` into an existing `OldTree` while preserving the BST ordering invariant, producing `NewTree`.

There are four distinct cases to consider:

1. **Base Case**: Inserting `N` into `nil` yields a new leaf node: `tree(N, nil, nil)`

   ```
   bst_ins(N, nil, tree(N, nil, nil)) :- !.
   ```

2. $N <$ Root: The element belongs in the **left subtree**. We recurse on `L` to obtain the updated subtree `Res1`, leaving `Root` and `R` unchanged

   ```
   bst_ins(N, tree(Root, L, R), tree(Root, Res1, R)) :-
       N < Root, !,
       bst_ins(N, L, Res1).
   ```

3. $N >$ Root: The element belongs in the **right subtree**. We recurse on `R` to obtain the updated subtree `Res1`, leaving `Root` and `L` unchanged

   ```
   bst_ins(N, tree(Root, L, R), tree(Root, L, Res1)) :-
       N > Root, !,
       bst_ins(N, R, Res1).
   ```

4. $N =$ Root (Duplicate element): If the element is already present at the root, we return the tree unchanged

   ```
   bst_ins(N, tree(N, L, R), tree(N, L, R)) :- !.
   ```

Let's test this, we want to start with the empty tree, insert 5, followed by 3 and 7:

```
?- bst_ins(5, nil, T1), bst_ins(3, T1, T2), bst_ins(7, T2, T3).
T1 = tree(5, nil, nil),
T2 = tree(5, tree(3, nil, nil), nil),
T3 = tree(5, tree(3, nil, nil), tree(7, nil, nil)).
```

BST Construction

Idea

Instead of inserting elements one by one, we want a predicate `bst_const/2` that takes a list of elements and gives us the binary search tree consisting of those elements

For example, we would like to get the final tree:

```
tree(5, tree(3, nil, nil), tree(7, nil, nil))
```

If we call that predicate with the list `[7,3,5]` instead of calling the insertion predicate three times:

```
bst_ins(5, nil, T1), bst_ins(3, T1, T2), bst_ins(7, T2, T3).
```

The base case is the empty list, we would just give `nil` back. For non-empty lists, we construct a BST for the tail, and then insert the head to the result we get.

```
bst_const([],nil).
bst_const([H|T], Res):-
    bst_const(T,T1),
    bst_ins(H,T1,Res).
```

```
?- bst_const([7,3,5],X).
X = tree(5,tree(3,nil,nil),tree(7,nil,nil))
```

Excursion: Tail Recursive BST Const.

We can instead write a tail-recursive predicate for efficiency. We will insert the elements left to right (starting from the head)

```
bst_const(X,Res):-
    bst_constAcc(X,nil,Res).

bst_constAcc([],Acc,Acc).
bst_constAcc([H|T],Acc,Res):-
    bst_ins(H,Acc,Acc1),
    bst_constAcc(T, Acc1, Res).
```

```
?- bst_const([5,3,7],X).
X = tree(5,tree(3,nil,nil),tree(7,nil,nil))
```

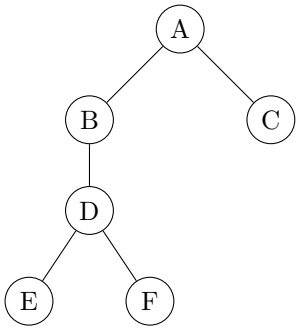
9 Tree Search Algorithms

To implement tree search algorithms we need to represent general trees in Prolog (not just binary trees).

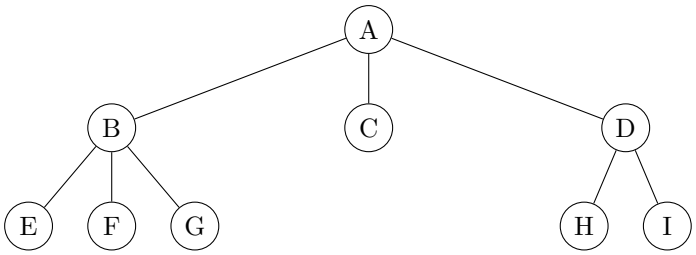
General Tree

In binary trees, we always knew that there is a left and right subtrees. But here we do not know how many children a node has.

Example 9.1: Tree Example



Example 9.2: Tree Example



Idea

A tree is a labelled node with a **list** of children, where each child can itself be a tree (with its own list of children) and leaves are just trees with no children (an empty list of children)

We create predicate `istree/1` to validate if something is a tree:

```
istree(tree(N, Children)):-
    string(N),
    istreelist(Children).

istreelist([]).
istreelist([H|T]):-
    istree(H),
    istreelist(T).
```

Let's represent the tree in Example 9.1

```
?- istree(
    tree("A",
         [ tree("B",
                [ tree("D",
                       [ tree("E", []),
                         tree("F", [])
                       ])
                ]),
           tree("C", [])
         ])
  ).
true
```

And the tree in Example 9.2

```
?- istree(
    tree("A",
         [ tree("B",
                [ tree("E", []),
                  tree("F", []),
                  tree("G", [])
                ]),
           tree("C", []),
           tree("D",
                [ tree("H", []),
                  tree("I", [])
                ])
         ])
  ).
true
```

DFS

We want to create a DFS predicate `dfs/1` that traverses the given tree in a pre-order DFS and prints out the traversed node label along with its depth (level). For example, if we give the tree in Example 9.2, we want to get:

```
0:A
1:B
2:E
2:F
2:G
1:C
1:D
2:H
2:I
true
```

Problem

1. How do we keep track of the current depth level D as we go deeper into the tree?
2. A node's `Children` is a **list** of unknown size. How do we traverse an arbitrary list of trees while maintaining depth-first order?

Idea

We want our predicate to be a public wrapper where it initially calls a helper predicate with initial depth of zero.

```
dfs(T) :-
    dfsD(T, 0).
```

Question

What should `dfsD/2` do?

At any node `tree(N, Children)` at depth `D`:

1. We first print the current node's label with its depth: `write(D), write(":"), writeln(N)`.
2. Its children will live one level deeper: `Next is D + 1`.

Problem

How do we traverse the list of children?

Idea

The Mutual Recursion Pattern (Trees $\longleftrightarrow$ Lists of Trees):

- A **tree** contains a list of children.
- A **list of children** contains individual trees.

This natural duality calls for **mutual recursion**: `dfsD` processes a single tree and delegates the children list to another predicate which walks through the list and calls `dfsD` on each child tree!

Therefore, `dfsD/2` will look like:

```
dfsD(tree(N, Children), D) :-
    write(D), write(":"), writeln(N),
    Next is D + 1,
    dfsAll(Children, Next).
```

Now, `dfsAll` simply walks down the list of sibling trees at depth `D`:

- **Base Case:** An empty list of children `[]` requires no action.
- **Recursive Step:** Given `[H|T]`, we process the first child tree `H` using `dfsD(H, D)`, and then recursively process the remaining sibling trees `T` using `dfsAll(T, D)`.

Observation

Notice a subtle but beautiful detail:

- When descending from parent to children in `dfsD`, depth increments: `Next is D + 1`.
- When moving across sibling trees in `dfsAll`, depth **remains the same** (`D`) because all siblings share the exact same level!

Putting things together:

```
% Public wrapper
dfs(T) :-
    dfsD(T, 0).

% Process a single node: print, increment depth, delegate children
dfsD(tree(N, Children), D) :-
    write(D), write(":"), writeln(N),
    Next is D + 1,
    dfsAll(Children, Next).

% Process a list of sibling trees
dfsAll([], _).
dfsAll([H|T], D) :-
    dfsD(H, D),
    dfsAll(T, D).
```

Let's run `dfs/1` on the tree from Example 9.2:

```
?- dfs(tree("A", [
           tree("B", [tree("E", []), tree("F", []), tree("G", [])]),
           tree("C", []),
           tree("D", [tree("H", []), tree("I", [])])
       ])).
0:A
1:B
2:E
2:F
2:G
1:C
1:D
2:H
2:I
true.
```

BFS

Similarly, we want to have a predicate `bfs/1` that traverses the tree in BFS (level by level). We are hoping to get the following output if we call it on the tree in Example 9.2:

```
0:A
1:B
1:C
1:D
2:E
2:F
2:G
2:H
2:I
true
```

Problem

In our depth-first traversal, Prolog did all the heavy lifting: its internal resolution engine naturally behaves like a LIFO stack. Descending into children was as simple as making a recursive call.
As we discussed earlier, BFS runs in direct opposition to Prolog's execution strategy! If we discover a node's children and immediately make recursive calls on them, Prolog will dive straight down the first child's subtree—collapsing back into DFS.
To explore nodes level by level, newly discovered children are not allowed to be visited right away. They must wait behind all previously discovered nodes. We need an explicit **FIFO (First-In, First-Out) queue**, traditionally called an **exploration fringe**.

Question

If our queue simply holds a list of trees waiting to be explored, say $[T_1, T_2, \dots]$, how will we know the depth of a node when we eventually pop it from the queue?

Note

Notice that at any intermediate stage, the queue might hold sibling nodes from depth 1 alongside newly enqueued children from depth 2. A single global counter or depth parameter passed to a recursive loop cannot work, because elements currently waiting in the queue belong to different depths!

Idea

We must **bundle each tree together with its own depth**!
Instead of storing bare tree terms in the queue, every element in our fringe will be an explicit pair:

$$(D,\ T)$$

where $D \in \mathbb{N}$ records the exact depth of subtree T. When search begins at the root, the queue starts with just a single pair:

$$[(0,\ \text{Root})]$$

Let us examine what a single step of breadth-first exploration looks like. Suppose our fringe is non-empty, with the first pair waiting at the front:

$$[(D,\ \text{tree}(N,\ \text{Children}))\ |\ \text{ToExplore}]$$

What actions must take place?

1. **Visit:** Dequeue the front pair and print its depth and label: $D : N$.
2. **Calculate Child Depth:** Since this node sits at depth D, all of its immediate children belong to depth $D + 1$.
3. **Tag Descendants:** Take the raw list of child trees and pair every single one of them with this new depth $D + 1$.
4. **Enqueue at the Back:** To preserve the FIFO discipline, add these newly tagged pairs to the **end** of the remaining queue (ToExplore).
5. **Recurse:** Continue exploring the updated queue.

When the queue eventually becomes empty (`[]`), every reachable node across all levels has been visited, and the process cleanly terminates.

Question

Before we assemble the main loop, we need a small helper to handle step (3).
Given a depth D and a list of child trees $[C_1, C_2, \dots, C_k]$, how do we construct a list of pairs $[(D, C_1), (D, C_2), \dots, (D, C_k)]$ in Prolog?

Solution

We define a pairing relation by structural induction on the list of children:

- **Base Case:** Pairing any depth with an empty list of children yields an empty list.
- **Recursive Step:** Given a head child H and tail T, we construct the pair (D, H) at the front, and recursively pair D with the remaining tail T.

In Prolog, this translates directly to:

```
pair(_, [], []).
pair(D, [H|T], [(D, H)|T2]) :-
    pair(D, T, T2).
```

With this helper ready, we can formulate our fringe exploration loop:

- **Base Case:** An empty fringe requires no work:

  ```
  bfsFringe([]).
  ```

- **Recursive Step:** Pull the front pair, print it, compute `Next is D + 1`, tag the children with `pair/3`, append them to the end of the existing queue, and recurse:

  ```
  bfsFringe([(D, tree(N, Children)) | ToExplore]) :-
      write(D), write(":"), writeln(N),
      Next is D + 1,
      pair(Next, Children, NextToExplore),
      append(ToExplore, NextToExplore, NewFringe),
      bfsFringe(NewFringe).
  ```

Finally, our `bfs/1` is just a wrapper that starts the fringe with the root at depth 0:

```
bfs(T) :-
    bfsFringe([(0, T)]).
```

Putting all the pieces together gives our complete breadth-first engine:

```
% wrapper
bfs(T) :-
    bfsFringe([(0, T)]).

% loop
% Base case: empty queue terminates
bfsFringe([]).

% Recursive step: pop front, print, tag children, append to back
bfsFringe([(D, tree(N, Children)) | ToExplore]) :-
    write(D), write(":"), writeln(N),
    Next is D + 1,
    pair(Next, Children, NextToExplore),
    append(ToExplore, NextToExplore, NewFringe),
    bfsFringe(NewFringe).

% Helper: tag each child tree with depth D
pair(_, [], []).
pair(D, [H|T], [(D, H)|T2]) :-
    pair(D, T, T2).
```

Example 9.3: BFS Queue Evolution Trace

To see the FIFO queue in action, let us trace how the fringe evolves when calling `bfs/1` on the tree from Example 9.2:

- **Step 0:** Fringe is $[(0, A)]$. Pop A, print `0:A`. Tag children B, C, D with depth 1 and enqueue them.
- **Step 1:** Fringe is $[(1, B), (1, C), (1, D)]$. Pop B, print `1:B`. Tag children E, F, G with depth 2 and append them to the back:

$$[(1, C),\ (1, D),\ (2, E),\ (2, F),\ (2, G)]$$

Notice that C and D remain in front of E, F, G!

- **Steps 2–3:** Pop and print `1:C` and `1:D`. Enqueue D's children H, I at depth 2.
- **Remaining Steps:** The queue now contains only depth 2 nodes: $[(2, E), (2, F), (2, G), (2, H), (2, I)]$, which are dequeued and printed in strict order.

Let's traverse the tree in Example 9.1:

```
?- bfs(tree("A",
          [ tree("B",
                 [ tree("D",
                        [ tree("E", []),
                          tree("F", [])
                        ])
                 ]),
            tree("C", [])
          ])).
0:A
1:B
1:C
2:D
3:E
3:F
true
```

DLS

Depth-Limited Search is a DFS but with a cut-off depth limit. We want to write a predicate `dls(T,L)` that performs DLS traversal of the tree T where L is the cut-off depth limit.

Example usage on the tree in Example 9.1 with depth limit $= 2$:

```
?- dls(tree("A",
          [ tree("B",
                 [ tree("D",
                        [ tree("E", []),
                          tree("F", [])
                        ])
                 ]),
            tree("C", [])
          ]),2).

0:A
1:B
2:D
1:C
true
```

Question

Think back to our standard DFS implementation. We used mutual recursion between a single-node processor and a list-of-children processor, carrying an accumulator D for the current depth:

- How do we inform our search of the limit L?
- When the search arrives at a node whose depth satisfies $D > L$, what should happen?

Idea

We must enforce a **guard condition**:

- If $D > L$, we have overstepped our allowed horizon. The search must immediately succeed vacuously without printing the node or expanding its children.
- If $D \leq L$, we proceed normally: print the node, increment the depth to $D + 1$, and explore its children under the same ceiling L.

Question

In Prolog, how do we express: "if $D > L$, stop and do nothing, and **never** attempt to fall through to the printing clause"?

We create a predicate `dfsUpTo`:

```
dfsUpTo(_, D, L) :-
    D > L,
    !.
```

If $D > L$ succeeds, the cut `!` commits immediately to this clause and burns any choice point that would otherwise attempt the subsequent clauses. Because the body ends right there, it succeeds quietly without printing anything or touching the children.

When $D \leq L$, we want to print the current node, compute the children's depth, and pass the list of children together with the next depth and `L` to our list processor. For that, we create the following predicate:

```
dfsUpTo(tree(N, Children), D, L) :-
    write(D), write(":"), writeln(N),
    Next is D + 1,
    dfsUpToAll(Children, Next, L).
```

The companion list processor simply distributes this logic across all siblings at depth D:

- **Base Case:** An empty list of children requires no actions:

  ```
  dfsUpToAll([], _, _).
  ```

- **Recursive Step:** Explore the head child, then recursively process the remaining siblings:

  ```
  dfsUpToAll([H|T], D, L) :-
      dfsUpTo(H, D, L),
      dfsUpToAll(T, D, L).
  ```

Finally, our public predicate simply receives the tree T and the cut-off limit L, initializing the search at the root level $D = 0$:

```
dls(T, L) :-
    dfsUpTo(T, 0, L).
```

Putting all the pieces together gives our complete Depth-Limited Search engine:

```
% Wrapper: start DLS at depth 0 with limit L
dls(T, L) :-
    dfsUpTo(T, 0, L).

% Cutoff clause: if depth exceeds limit, prune branch
dfsUpTo(_, D, L) :-
    D > L,
    !.

% Expansion clause: visit node, increment depth, recurse into children
dfsUpTo(tree(N, Children), D, L) :-
    write(D), write(":"), writeln(N),
    Next is D + 1,
    dfsUpToAll(Children, Next, L).

% Process list of siblings: base case
dfsUpToAll([], _, _).

% Process list of siblings: recursive step
dfsUpToAll([H|T], D, L) :-
    dfsUpTo(H, D, L),
    dfsUpToAll(T, D, L).
```

Let us test `dls/2` on the tree from Example 9.2 with different depth limits L.

Setting $L = 1$ cuts off all children at depth 2 (E, F, G, H, I):

```
?- dls(tree("A", [
            tree("B", [tree("E", []), tree("F", []), tree("G", [])]),
            tree("C", []),
            tree("D", [tree("H", []), tree("I", [])])
        ]), 1).
0:A
1:B
1:C
1:D
true.
```

If we set $L = 0$, only the root is visited:

```
?- dls(tree("A", [
            tree("B", [tree("E", []), tree("F", []), tree("G", [])]),
            tree("C", []),
            tree("D", [tree("H", []), tree("I", [])])
        ]), 0).
0:A
true.
```

Note

Setting $L = 2$ allows the search to reach the full depth, reproducing the complete DFS traversal.

IDS

Iterative Deepening Search is a DLS with an increasing depth limit. We want a predicate `ids/1` that traverses the tree starting with a depth limit 0 and increasing until the maximum actual depth of the tree. For each depth limit, we want to print that depth limit and then traverse the nodes as usual.

Here is what we hope for if we give the tree in Example 9.1

```
?- ids(tree("A",
          [ tree("B",
                 [ tree("D",
                        [ tree("E", []),
                          tree("F", [])
                        ])
                 ]),
            tree("C", [])
          ])).

LIMIT = 0
0:A
LIMIT = 1
0:A
1:B
1:C
LIMIT = 2
0:A
1:B
2:D
1:C
LIMIT = 3
0:A
1:B
2:D
3:E
3:F
1:C
true
```

Idea

Just call DLS with increasing depth limits starting from 0.

Problem

We want to know when to stop. Because for a finite tree (such as Example 9.1, which has a maximum depth of 3) once we run DLS with $L = 3$, the entire tree has already been completely explored. If we naively continue with $L = 4, 5, 6, \ldots$, it would endlessly re-traverse the exact same tree!

Idea

We want some way of reporting back to the outer lopp that the entire tree was explored

Idea

We can equip our depth-limited traversal with a **termination flag**:

$$\text{Done} \in \{0, 1\}$$

- If at **any** point during the traversal a node exceeds the depth limit ($D > L$), that branch was cut off. Thus, the tree was *not* fully explored, so we report Done $= 0$.
- If a node is a leaf (empty children list `[]`), it has no children to cut off, so that branch is fully explored: Done $= 1$.
- For an internal node with multiple children, the entire subtree is fully explored if and only if **all** of its children are fully explored! In arithmetic, a logical AND over binary flags is simply multiplication:

  $$\text{Done} = \text{Done}_H \times \text{Done}_T$$

  If even a single child was truncated (returning 0), the product collapses to 0!

Let us refine our depth-limited traversal to carry this fourth argument, reflecting our three cases:

1. **Cutoff Hit ($D > L$):** When the depth ceiling is breached, we prune the branch with a cut (`!`) and unify the status flag with 0:

```
dfsUpTo(_, D, L, 0) :-
    D > L,
    !.
```

2. **Node Expansion ($D \leq L$):** When within limits, we print the node, increment the depth, and let the list of children determine the overall `Done` status:

```
dfsUpTo(tree(N, Children), D, L, Done) :-
    write(D), write(":"), writeln(N),
    D1 is D + 1,
    dfsUpToAll(Children, D1, L, Done).
```

3. **Children Exploration:** An empty list of children represents an exhausted branch (Done = 1). A non-empty list computes the conjunction of all child flags via multiplication:

```
dfsUpToAll([], _, _, 1).
dfsUpToAll([H|T], D, L, Done) :-
    dfsUpTo(H, D, L, DoneH),
    dfsUpToAll(T, D, L, DoneT),
    Done is DoneH * DoneT.
```

Question

Now that our traversal returns Done $\in \{0, 1\}$, how do we structure the outer deepening loop?

Solution

At each stage with current limit C:

1. Print `LIMIT = C`.
2. Launch the traversal: `dfsUpTo(T, 0, C, Done)`.
3. Inspect the result using a helper predicate:
   - If Done = 1, the entire tree was explored without hitting any limit ceiling. Search is complete!
   - If Done = 0, at least one branch was truncated. Increment the limit to $C + 1$ and recurse!

In Prolog, this logic is expressed declaratively with two clauses:

```
% Base case: entire tree explored without cutoffs -> stop!
handleDone(_, _, 1).

% Recursive step: cutoff occurred -> increment limit and repeat
handleDone(T, C, 0) :-
    C1 is C + 1,
    idsUntilDone(T, C1).
```

Our looping predicate ties everything together:

```
idsUntilDone(T, C) :-
    write('LIMIT = '), writeln(C),
    dfsUpTo(T, 0, C, Done),
    handleDone(T, C, Done).
```

Finally, the public entry point starts the iterative deepening process with limit 0:

```
ids(T) :-
    idsUntilDone(T, 0).
```

Putting all the components together gives our complete Iterative Deepening Search engine:

```
% Wrapper: start IDS at limit 0
ids(T) :-
    idsUntilDone(T, 0).

% Outer loop
idsUntilDone(T, C) :-
    write('LIMIT = '), writeln(C),
    dfsUpTo(T, 0, C, Done),
    handleDone(T, C, Done).

% Done = 1 -> fully explored, stop
handleDone(_, _, 1).

% Done = 0 -> branch cut off, deepen limit
handleDone(T, C, 0) :-
    C1 is C + 1,
    idsUntilDone(T, C1).

% depth exceeded -> branch truncated (Done = 0)
dfsUpTo(_, D, L, 0) :-
    D > L,
    !.

% visit node, delegate Done status to children
dfsUpTo(tree(N, Children), D, L, Done) :-
    write(D), write(":"), writeln(N),
    D1 is D + 1,
    dfsUpToAll(Children, D1, L, Done).

% no children -> fully explored (Done = 1)
dfsUpToAll([], _, _, 1).

% Done is logical AND of all child statuses
dfsUpToAll([H|T], D, L, Done) :-
    dfsUpTo(H, D, L, DoneH),
    dfsUpToAll(T, D, L, DoneT),
    Done is DoneH * DoneT.
```

10 MiniMax

Minimax is an adversarial search algorithm where the agent competes against an opponent whose goals directly conflict with its own.

We consider **two-player, zero-sum games of perfect information**:

- **MAX** (Player $+1$): seeks to maximize the terminal game utility.
- **MIN** (Player -1): seeks to minimize the terminal game utility.

Assuming both players play with perfect rationality, the game-theoretic value $V(s)$ of any state s is governed by the **Minimax equation**:

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is a terminal state,} \\ \max_{s' \in \text{Children}(s)} V(s') & \text{if it is MAX's turn to move,} \\ \min_{s' \in \text{Children}(s)} V(s') & \text{if it is MIN's turn to move.} \end{cases}$$

To make this concrete, we implement an ancient mathematical game: the **Subtraction Game** (a single-heap variant of Nim):

- A pile contains N matches.
- Two players alternate turns. On their turn, a player must remove 1, 2, or 3 matches from the pile (provided enough matches remain).
- A player who faces an empty pile ($N = 0$) has no legal moves and **loses**. Equivalently, the player who removes the last match wins!

Idea

State Representation: We represent the board state as a compound term:

$$\texttt{state}(N,\ P)$$

where $N \in \mathbb{N}$ denotes the number of remaining matches, and $P \in \{1,\ -1\}$ denotes the active player whose turn it is.

Note

Switching turns is simply arithmetic negation.

Step 1: Generating Successor States

From a state $\texttt{state}(N, P)$, what moves are legally available?

- If $N \geq 1$, the player can remove 1 match, leaving $N - 1$ matches for player $-P$.
- If $N \geq 2$, the player can remove 2 matches, leaving $N - 2$ matches for player $-P$.
- If $N \geq 3$, the player can remove 3 matches, leaving $N - 3$ matches for player $-P$.

When $N = 0$, no matches remain, and no legal transitions exist.

In Prolog, we represent these non-deterministic possibilities with three distinct clauses:

```
% Remove 1 match
successor(state(N, P), NextS) :-
    N > 0,
    N2 is N - 1,
    P2 is -P,
    NextS = state(N2, P2).

% Remove 2 matches
successor(state(N, P), NextS) :-
    N > 1,
    N2 is N - 2,
    P2 is -P,
    NextS = state(N2, P2).

% Remove 3 matches
successor(state(N, P), NextS) :-
    N > 2,
    N2 is N - 3,
    P2 is -P,
    NextS = state(N2, P2).
```

Step 2: Collecting All Legal Moves

To compute max or min, our search needs the **complete list** of all valid child states originating from the current state.

Question

How can we collect all non-deterministic solutions of `successor/2` into a single Prolog list?

Solution

We use an accumulator pattern coupled with negation-as-failure:

- **Recursive Step:** Generate a successor `NextS` using `successor(S, NextS)`. Check that it has not already been added to our accumulator: `\+ is_member(NextS, Acc)`. Prepend it to the accumulator and recurse. We cut to commit to collecting all remaining successors.
- **Base Case:** When no new unvisited successors can be generated, the accumulator holds the complete set of moves.

```
successors(S, Acc, NextSs) :-
    successor(S, NextS),
    \+ is_member(NextS, Acc),
    successors(S, [NextS|Acc], NextSs),
    !.
successors(_, Acc, Acc).
```

Step 3: Finding Extremes (Min and Max)

Question

Once MAX (or MIN) obtains the list of reachable child states, how do they determine the optimal value?

Idea

We fold over the list of child states while maintaining an accumulator for the best utility seen so far:

- **MAX's perspective (`maxvalue/3`):** For each child state, compute its Minimax value. If the child's value is greater than or equal to the current maximum, update the running maximum.
- **MIN's perspective (`minvalue/3`):** For each child state, compute its Minimax value. If the child's value is strictly less than the current minimum, update the running minimum.

Here are the dual fold predicates:

```
% maxvalue/3: Find the maximum value among child states
maxvalue([], BestSoFar, BestSoFar).

maxvalue([State|Rest], BestSoFar, Result) :-
    value(State, ChildValue),
    ChildValue >= BestSoFar,
    maxvalue(Rest, ChildValue, Result).

maxvalue([State|Rest], BestSoFar, Result) :-
    value(State, ChildValue),
    ChildValue < BestSoFar,
    maxvalue(Rest, BestSoFar, Result).

% minvalue/3: Find the minimum value among child states
minvalue([], BestSoFar, BestSoFar).

minvalue([State|Rest], BestSoFar, Result) :-
    value(State, ChildValue),
    ChildValue < BestSoFar,
    minvalue(Rest, ChildValue, Result).

minvalue([State|Rest], BestSoFar, Result) :-
    value(State, ChildValue),
    ChildValue >= BestSoFar,
    minvalue(Rest, BestSoFar, Result).
```

Step 4: The Minimax Engine and Terminal States

Now we tie everything together in our main evaluation predicate:

- If it is **MAX's turn** ($P = 1$): collect all child states, and find the maximum starting from the worst-case utility -1.
- If it is **MIN's turn** ($P = -1$): collect all child states, and find the minimum starting from the worst-case utility $+1$.

```
value(State, Value) :-
    State = state(_, Player),
    Player = 1,
    successors(State, [], ChildStates),
    maxvalue(ChildStates, -1, Value).

value(State, Value) :-
    State = state(_, Player),
    Player = -1,
    successors(State, [], ChildStates),
    minvalue(ChildStates, 1, Value).
```

Observation

Suppose we reach a terminal state with no matches left:

- **State** `state`$(0, 1)$**:** It is MAX's turn, but $N = 0$. `successors/3` generates an empty list `ChildStates` = []. Then `maxvalue([], -1, Value)` immediately unifies `Value = -1`! MAX has no legal moves and therefore loses.
- **State** `state`$(0, -1)$**:** It is MIN's turn, but $N = 0$. `ChildStates` is empty. Then `minvalue([], 1, Value)` unifies `Value = 1`! MIN has no legal moves and loses, meaning MAX wins!

Example 10.1

Let us query `value/2` for various starting pile sizes with Player 1 (MAX) moving first:

```
?- value(state(1, 1), V).
V = 1.

?- value(state(2, 1), V).
V = 1.

?- value(state(3, 1), V).
V = 1.

?- value(state(4, 1), V).
V = -1.
```

Mathematical Explanation:

- For $N \in \{1, 2, 3\}$, MAX can immediately take all remaining matches, reducing the pile to 0 and leaving MIN with no moves ($V = 1$).
- For $N = 4$, whatever move MAX chooses ($1, 2$, or 3), MIN is left with $3, 2$, or 1 matches and can take the rest to win on the very next turn! Thus, $N = 4$ is a guaranteed losing state for the first player ($V = -1$).

In general, for any subtraction game with moves $\{1, 2, 3\}$, a position is a losing position for the current player if and only if:

$$N \equiv 0 \pmod 4$$

Our Minimax implementation discovers this exact mathematical truth automatically through search!

11 Fun with Propositional Logic

A propositional well-formed formula (WFF) is defined inductively over a signature of variable symbols $\mathcal{V}$ and logical connectives $\{\neg, \wedge, \vee, \rightarrow\}$:

- Every propositional variable $p \in \mathcal{V}$ is a formula.
- If φ is a formula, then $\neg\varphi$ is a formula.
- If φ and ψ are formulas, then $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, and $(\varphi \rightarrow \psi)$ are formulas.

We want to write a predicate `isForm(L,F)` that checks if F is a well-formed formula given the list of variables L.

```
isForm(L, var(X)) :-
    string(X),
    is_member(X, L).

isForm(L, neg(X)) :-
    isForm(L, X).

isForm(L, disj(X, Y)) :-
    isForm(L, X),
    isForm(L, Y).

isForm(L, conj(X, Y)) :-
    isForm(L, X),
    isForm(L, Y).

isForm(L, impl(X, Y)) :-
    isForm(L, X),
    isForm(L, Y).
```

Let's test this:

```
% Valid negation
?- isForm(["a", "b"], neg(var("a"))).
true.

% Variable 'c' is not in the declared alphabet ["a", "b"]
?- isForm(["a", "b"], neg(var("c"))).
false.

% Arity mismatch: impl requires 2 arguments
?- isForm(["a", "b"], conj(var("a"), impl(var("b")))).
false.

% Nested compound formula
?- isForm(["p", "q", "r"], impl(var("p"), disj(var("q"), var("r")))).
true.
```

Simplify

In propositional logic, the main connectives are $\neg$ and $\wedge$. Every other connectives can be written using negation and conjunction.

$$A \vee B \equiv \neg(\neg A \wedge \neg B)$$

$$A \Rightarrow B \equiv \neg A \vee B \equiv \neg(A \wedge \neg B)$$

We want a predicate `simplify(Formula, Simplified)` that takes an arbitrary formula and transforms it into an equivalent formula containing only `var`, `neg`, and `conj`.

```
% Base Case: a variable is already simplified
simplify(var(X),var(X)).

% A negation of something is simplified if that something is simplified
simplify(neg(X), neg(Y)):-simplify(X,Y).

% A conjunction of two things is simplified if the two things are
simplify(conj(X,Y),conj(X1,Y1)):-simplify(X,X1), simplify(Y,Y1).

% Disjunction
simplify(disj(X,Y),neg(conj(neg(X1),neg(Y1)))):-
    simplify(X,X1),
    simplify(Y,Y1).

% Implication
simplify(impl(X,Y), neg(conj(X1, neg(Y1)))):-
    simplify(X,X1),
    simplify(Y,Y1).
```

Let's test this:

```
?- simplify(disj(var("a"), var("b")), X).
X = neg(conj(neg(var("a")), neg(var("b")))).

?- simplify(impl(var("a"), var("b")), X).
X = neg(conj(var("a"), neg(var("b")))).
```

Evaluation

Now we turn to **semantics**: computing the truth value of a formula under a given variable assignment (valuation) $\nu : \mathcal{V} \to \{0, 1\}$.

We represent an assignment as a list of key-value associations:

$$[\texttt{assign("a", 1)}, \texttt{assign("b", 0)}]$$

We want a predicate `eval(Assignments, Formula, Value)` where Value $\in \{0, 1\}$.

Question

How do we evaluate logical operators cleanly in Prolog without writing multi-clause truth tables for every single connective?

Algebraic Encoding of Boolean Logic

$$\begin{aligned}
\nu(\neg A) &= 1 - \nu(A) \\
\nu(A \wedge B) &= \nu(A) \times \nu(B) \\
\nu(A \vee B) &= \nu(A) + \nu(B) - (\nu(A) \times \nu(B)) \\
\nu(A \Rightarrow B) &= \nu(\neg A \vee B) = (1 - \nu(A)) + \nu(B) - ((1 - \nu(A)) \times \nu(B))
\end{aligned}$$

Sanity Check - Disjunction

- $0 \vee 0 = 0 + 0 - 0 = \mathbf{0}$
- $0 \vee 1 = 0 + 1 - 0 = \mathbf{1}$
- $1 \vee 0 = 1 + 0 - 0 = \mathbf{1}$
- $1 \vee 1 = 1 + 1 - 1 = \mathbf{1}$

Idea

We use this algebraic encoding to evaluate formulas!

```
% Base Case (variable) -> just a lookup
eval(L, var(X), Y) :-
    is_member(assign(X, Y), L).

% Negation
eval(L, neg(X), Z) :-
    eval(L, X, V),
    Z is 1 - V.

% Conjunction
eval(L, conj(X, Y), V) :-
    eval(L, X, X1),
    eval(L, Y, Y1),
    V is X1 * Y1.

% Disjunction
eval(L, disj(X, Y), V) :-
    eval(L, X, X1),
    eval(L, Y, Y1),
    Plus is X1 + Y1,
    Mult is X1 * Y1,
    V is Plus - Mult.

% Implication
eval(L, impl(X, Y), V) :-
    eval(L, X, X1),
    eval(L, Y, Y1),
    NotX is 1 - X1,
    Plus is NotX + Y1,
    Mult is NotX * Y1,
    V is Plus - Mult.
```

Let's test this:

```
?- eval([assign("a", 1), assign("b", 0)], conj(var("a"), var("b")), V).
V = 0.

?- eval([assign("p", 1), assign("q", 0)], impl(var("p"), var("q")), V).
V = 0.

?- eval([assign("p", 0), assign("q", 0)], impl(var("p"), var("q")), V).
V = 1.

?- eval([assign("a", 0), assign("b", 0), assign("c", 1)],
        conj(neg(var("a")), disj(var("b"), var("c"))), V).
V = 1.
```

12 Theoretical Foundations

12.1 Logical Systems and First-Order Logic

Def 12.1. Logical System: A **logical system** (or simply a **logic**) is a triple $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$, where

- $\mathcal{L}$ is a set of propositions
- $\mathcal{M}$ is a set of **models**,
- $\models$ is a relation $\models \subseteq \mathcal{M} \times \mathcal{L}$ called the **satisfaction relation**. We read $\mathcal{M} \models A$ as $\mathcal{M}$ **satisfies** A and correspondingly $\mathcal{M} \not\models A$ as $\mathcal{M}$ **falsifies** A

Def 12.2. Let $\langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, $M \in \mathcal{M}$ a model and $A \in \mathcal{L}$ a proposition. Then we say that A is

- **satisfied** by M iff $M \models A$
- **satisfiable** iff A is satisfied by some model
- **unsatisfiable** iff A is not satisfiable
- **falsified** by M iff $M \not\models A$
- **valid** or **unfalsifiable** (write $\models A$) iff A is satisfied by **every** model $M \in \mathcal{M}$
- **invalid** or **falsifiable** (write $\not\models A$) iff A is not valid

Def 12.3. Derivation Relation ($\vdash$): Let $\mathcal{L}$ be a formal language. For any set of propositions $\mathcal{H} \subseteq \mathcal{L}$ (*called the context or hypotheses*) and any proposition $A \in \mathcal{L}$, we write $\mathcal{H} \vdash A$ to say that A is derivable from $\mathcal{H}$. We call a relation $\vdash \subseteq \mathcal{P}(\mathcal{L}) \times \mathcal{L}$ a **derivation relation** for $\mathcal{L}$, iff

- $\mathcal{H} \vdash A$, if $A \in \mathcal{H}$ (proof reflexive),
- $\mathcal{H} \vdash A$ and $(\mathcal{H}' \cup \{A\}) \vdash B$ imply $(\mathcal{H} \cup \mathcal{H}') \vdash B$ (proof transitive),
- $\mathcal{H} \vdash A$ and $\mathcal{H} \subseteq \mathcal{H}'$ imply $\mathcal{H}' \vdash A$ (monotonic admits weakening).

Def 12.4. Inference Rules: Let $\mathcal{L}$ be a formal language, then an **inference rule** over $\mathcal{L}$ is decidable $n+1$-ary relation on $\mathcal{L}$. Inference rules are traditionally written as:

$$\frac{A_1 \quad \ldots \quad A_n}{C}\ \mathcal{N}$$

where $A_1, \cdots, A_n$ and C are schemata for words in $\mathcal{L}$ and $\mathcal{N}$ is a name. The A_i are called **assumptions** of $\mathcal{N}$, and C is called its **conclusion**.

Any $n+1$-tuple $\frac{a_1 \quad \ldots \quad a_n}{c}$ in $\mathcal{N}$ is called an **application** of $\mathcal{N}$ and we say that we apply $\mathcal{N}$ to a set M of words with $a_1, \cdots, a_n \in M$ to obtain c.

Def 12.5. Axiom: An inference rule without assumptions is called an **axiom** (`Ax`)

Def 12.6. Calculus: A **calculus** (or **inference system**) is a formal language $\mathcal{L}$ equipped with a set $\mathcal{C}$ of inference rules over $\mathcal{L}$.

Def 12.7. $\mathcal{C}$-derivation: Let $S := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, and $\mathcal{C}$ a calculus for $\mathcal{L}$, then a $\mathcal{C}$-derivation of a proposition $P \in \mathcal{L}$ from a set $\mathcal{H} \subseteq \mathcal{L}$ of hypotheses (write $\mathcal{H} \vdash_{\mathcal{C}} P$) is a sequence $A_1, \cdots, A_m$ of propositions where:

- $A_m = P$,
- for all $1 \leq i \leq m$, either $A_i \in \mathcal{H}$, or,
- there is an inference rule $\frac{A_{l_1} \quad \ldots \quad A_{l_k}}{A_i}\ \mathcal{N}$ in $\mathcal{C}$ with $l_j < i$ for all $j \leq k$

We can also see a $\mathcal{C}$-derivation as a **derivation tree**, where A_{l_j} are the children of the node A_i

Def 12.8. $\mathcal{C}$-refutation: Let $\mathcal{C}$ be a calculus, then a formula set Φ is called **$\mathcal{C}$-refutable** if there is a **$\mathcal{C}$-refutation**, i.e. a $\mathcal{C}$-derivation of a contradiction from Φ. The act of finding a **refutation** for Φ is called **refuting** Φ

Def 12.9. Soundness: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system and $\mathcal{H} \subseteq \mathcal{L}$ a set of hypotheses, then we call a calculus $\mathcal{C}$ over $\mathcal{L}$ **sound** (correct), iff $\mathcal{H} \models A$ whenever $\mathcal{H} \vdash_{\mathcal{C}} A$

Def 12.10. Completeness: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system and $\mathcal{H} \subseteq \mathcal{L}$ a set of hypotheses, then we call a calculus

$\mathcal{C}$ over $\mathcal{L}$ **complete**, iff $\mathcal{H} \vdash_{\mathcal{C}} A$ whenever $\mathcal{H} \models A$

Def 12.11. Formal System: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, and let $\mathcal{C}$ be a calculus over the language $\mathcal{L}$. Then the pair $\langle \mathcal{S}, \mathcal{C} \rangle$ is called a **formal system**.

Def 12.12. Theorem Proving: Given a formal system $\langle \mathcal{S}, \mathcal{C} \rangle$, where $\mathcal{C}$ is a calculus and $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ is a logical system, the task of **theorem proving** is the task of determining whether $\mathcal{H} \vdash_{\mathcal{C}} C$ for a **conjecture** $C \in \mathcal{L}$ and hypotheses $\mathcal{H} \subseteq \mathcal{L}$

Def 12.13. Prolog uses a simple calculus consisting of the following inference rules to build the knowledge base:

- Modus Ponens (MP)
- Conjunction Introduction ($\wedge I$)
- Substitution (`Subst`)

$$\frac{A \Rightarrow B \quad A}{B}\ \text{MP} \qquad \frac{A \quad B}{A \wedge B}\ \wedge I \qquad \frac{A}{[B/X](A)}\ \texttt{Subst}$$

Def 12.14. Propositional Logic: Propositional Logic (PL^0) is a logical system $\langle \mathcal{L}, \mathcal{M}, \models \rangle$ Where

- the formal language $\mathcal{L}$ is defined as the well-formed formulae of propositional logic: $\text{wff}_0(\Sigma_{\text{PL}^0}, \mathcal{V}_{\text{PL}^0})$,
- the set $\mathcal{M}$ of models is given by the set $\mathcal{K}_0$ of total functions $\varphi : \mathcal{V}_0 \to \mathcal{D}_0$ (i.e., propositional variable assignments),
- and $\varphi \models A$ iff $\mathcal{I}_\varphi(A) = T$ (a proposition is satisfiable by model φ if the interpretation of it -under that φ- is true)

Hence, propositional logic is a logical system $\langle \text{wff}_0(\Sigma_{\text{PL}^0}, \mathcal{V}_{\text{PL}^0}), \mathcal{K}_0, \models \rangle$

Def 12.15. Formulae of PL^0: The formulae of PL^0 are made up from

- propositional variables $\mathcal{V}_{\text{PL}^0}$ (or just $\mathcal{V}_0$) : $\mathcal{V}_0 := \{P, Q, R, \cdots\}$ (countably infinite)
- propositional signature Σ_{PL^0} (or just Σ_0): $\Sigma_0 := \{\top, \bot, \neg, \vee, \wedge, \Rightarrow, \Leftrightarrow, \cdots\}$ (called *connectives*)

We define the set $\text{wff}_0(\Sigma_{\text{PL}^0}, \mathcal{V}_{\text{PL}^0})$ of **well-formed propositional formulae (wffs)** as:

- propositional variables
- logical constants $\top$ and $\bot$
- negations $\neg A$
- conjunctions $A \wedge B$
- disjunctions $A \vee B$
- implications $A \Rightarrow B$, and
- equivalence (biimplications) $A \Leftrightarrow B$

where $A, B \in \text{wff}_0(\Sigma_{\text{PL}^0}, \mathcal{V}_{\text{PL}^0})$ themselves.

Def 12.16. PL^1: First-Order (Predicate) Logic PL^1 is a formal system that combines propositional logic (PL^0) with the ability to quantify over individuals

Def 12.17. First-Order Signature: A First-Order Signature is a tuple $\Sigma := \langle \Sigma_1, \Sigma_0 \rangle$ where $\Sigma_1 := \Sigma^f \cup \Sigma^p \cup \Sigma^{sk}$:

- $\Sigma^f := \bigcup_{k \in \mathbb{N}} \Sigma^f_k$ of **function constants** (or **terms**), where members of Σ^f_k denote the k-ary functions on **individuals**
- $\Sigma^p := \bigcup_{k \in \mathbb{N}} \Sigma^p_k$ of **predicate constants**, where members of Σ^p_k denote k-ary relations among individuals
- $\Sigma^{sk} := \bigcup_{k \in \mathbb{N}} \Sigma^{sk}_k$ of **Skolem constants** that act as witness constructors
- All are pairwise disjoint, countable sets of symbols for each $k \in \mathbb{N}$.

We also assume a set of **individual variables** $V_\iota := \{X, Y, Z, \cdots\}$ which we can quantify over them.

Def 12.18. PL^1 Terms: The set $\text{wff}_\iota(\Sigma_1, V_\iota)$ of **well-formed terms** over the signature Σ_1 and variables V_ι is defined by the following grammar:

$$\begin{array}{rcl} f^k & \in & \Sigma^f_k \cup \Sigma^{sk}_k \\ X & \in & V_\iota \\ t & ::= & X \mid f^k(t_1, \cdots, t_k) \end{array}$$

Def 12.19. PL^1 Propositions: The set $\mathrm{wff}_\circ(\Sigma_1, V_\iota)$ of **well-formed formulae** over the signature Σ_1 and variables V_ι is defined by the following grammar:

$$\begin{array}{rcl} p^k & \in & \Sigma_k^p \\ t_i & \in & \mathrm{wff}_\iota(\Sigma_1, V_\iota) \\ X & \in & V_\iota \\ A & ::= & p^k(t_1, \cdots, t_k) \mid \top \mid \neg A \mid A \wedge A \mid \forall X.A \end{array}$$

Def 12.20. Free Variables: An occurrence of a variable X is **free** in a formula A if it is not within the scope of a binder. We define the set of free variables in A ($\mathrm{free}(A)$) recursively as follows:

- **For Terms** ($t \in \mathrm{wff}_\iota(\Sigma_1, V_\iota)$)
  - $\mathrm{free}(X) = \{X\}$
  - $\mathrm{free}(f^k(t_1, \cdots, t_k)) := \bigcup_{i=1}^k \mathrm{free}(t_i)$
- **For Formulae** ($A \in \mathrm{wff}_\circ(\Sigma_1, V_\iota)$)
  - $\mathrm{free}(p^k(t_1, \cdots, t_k)) := \bigcup_{i=1}^k \mathrm{free}(t_i)$
  - $\mathrm{free}(\top) := \emptyset$
  - $\mathrm{free}(\neg A) := \mathrm{free}(A)$
  - $\mathrm{free}(A \wedge B) := \mathrm{free}(A) \cup \mathrm{free}(B)$
  - $\mathrm{free}(\forall X.A) := \mathrm{free}(A) \setminus \{X\}$

Def 12.21. Bound Variables: An occurrence of a variable X is **bound** if it occurs within the scope of a quantifier. We define the set of bound variables $\mathrm{BVar}(A)$ as the set of all variables that are *captured* by a binder:

- **For Terms** ($t \in \mathrm{wff}_\iota(\Sigma_1, V_\iota)$)
  - $\mathrm{BVar}(t) := \emptyset$ (*variables in terms are always free until a formula provides a binder*)
- **For Formulae** ($A \in \mathrm{wff}_\circ(\Sigma_1, V_\iota)$)
  - $\mathrm{BVar}(p^k(t_1, \cdots, t_k)) := \emptyset$
  - $\mathrm{BVar}(\neg A) := \mathrm{BVar}(A)$
  - $\mathrm{BVar}(A \wedge B) := \mathrm{BVar}(A) \cup \mathrm{BVar}(B)$
  - $\mathrm{BVar}(\forall X.A) := \mathrm{BVar}(A) \cup \{X\}$

Def 12.22. Ground Formulae: A PL^1 formula A is called **ground/closed** iff $\mathrm{free}(A) := \emptyset$. A closed formula is called a sentence. We denote the set of all **ground** terms with $\mathrm{cwff}_\iota(\Sigma_1, V_\iota)$ and the set of all sentences with $\mathrm{cwff}_\circ(\Sigma_1, V_\iota)$

Def 12.23. Alphabetical Variants: Bound variables can be **renamed**; specifically, if $\forall X.B$ is a sub-formula of A, let A' be the result of replacing $\forall X.B$ in A with $\forall Y.B'$, where B' is obtained from B by replacing all occurrences of $X \in \mathrm{free}(B)$ with a new variable Y that does not occur in A. We call A' an **alphabetical variant** of A (and vice versa).

Def 12.24. Expression Language: Let signature Σ and variables $\mathcal{V}$ be disjoint sets and G a grammar whose vocabulary is $\Sigma \cup \mathcal{V}$, then we call $\langle \Sigma, \mathcal{V}, G \rangle$ an **expression language**, and denote the language of G with $\mathrm{wfe}(\Sigma, \mathcal{V})$: the set of **well-formed expressions**.

Note

For PL^1, we have:

$$\mathrm{wfe}(\Sigma_1, \mathcal{V}_\iota) := \mathrm{wff}_\circ(\Sigma_1, V_\iota) \cup \mathrm{wff}_\iota(\Sigma_1, V_\iota)$$

12.2 Substitutions and Unification

Def 12.25. Support: Let $f : A \to B$ be a function. The **support** of f, denoted by $\mathrm{supp}(f)$, is the set of all elements in the domain A for which the function does not return a "default" or "identity" value.

Example 12.1: Default is Zero

Let $f : \mathbb{Z} \to \mathbb{Z}$ be a function that assigns almost every integer to 0, except:

$$f(2) = 5, \quad f(7) = -3, \quad \text{and} \quad f(x) = 0 \text{ for all other } x \in \mathbb{Z}.$$

The "default" value is 0. Therefore, the support consists only of the points that deviate from 0:

$$\text{supp}(f) = \{2, 7\}$$

Example 12.2: Default is Identity

Consider a variable mapping σ over an infinite set of variables $\mathcal{V} = \{X, Y, Z, W, \dots\}$ where the default behavior is to leave a variable unchanged ($\sigma(V) = V$).
If σ only modifies two variables:

$$\sigma(X) = a, \quad \sigma(Y) = g(Z), \quad \text{and} \quad \sigma(V) = V \text{ for all other } V \in \mathcal{V},$$

then the support of σ is precisely the finite set of variables whose values actually change:

$$\text{supp}(\sigma) = \{X, Y\}$$

Def 12.26. Substitution: Let $\text{wfe}(\Sigma, \mathcal{V})$ be an expression language, then we call $\sigma : \mathcal{V} \to \text{wfe}(\Sigma, \mathcal{V})$ a **substitution** iff the support $\text{supp}(\sigma) := \{X \mid (X, A) \in \sigma \wedge X \neq A\}$ of σ is finite. We denote the **empty substitution** with ϵ.

Def 12.27. We can **discharge** a variable X from a substitution σ by setting $\sigma_{-X} := \sigma, [X/X]$

Def 12.28. Introduced Variables: Let σ be a substitution. The set of variables **introduced** by σ, denoted by $\text{intro}(\sigma)$, is the set of all free variables occurring in the replacement terms:

$$\text{intro}(\sigma) := \bigcup_{X \in \text{supp}(\sigma)} \text{free}(\sigma(X))$$

Example 12.3

consider the formula $A := \forall X.p(X, Y)$ and the substitution $\sigma := \{[Z/Y]\}$:

- $\text{supp}(\sigma) = \{Y\}$
- $\text{free}(\sigma(Y)) = \text{free}(Z) = \{Z\}$
- $\text{intro}(\sigma) = \text{free}(\sigma(Y)) = \{Z\}$

Def 12.29. Substitution Application: We define substitution application by:

- $\sigma(c) = c$ for $c \in \Sigma$
- $\sigma(X) = A$ iff $X \in \mathcal{V}$ and $(X, A) \in \sigma$
- $\sigma(f(A_1, \cdots, A_n)) = f(\sigma(A_1), \cdots, \sigma(A_n))$
- $\sigma(\forall X.A) = \forall X.\sigma_{-X}(A)$

Example 12.4

consider the formula $g(x, y, h(z))$, and we want to apply the substitution $\sigma := \{x \mapsto a, y \mapsto f(b), z \mapsto a\}$ we get the formula $g(a, f(b), h(a))$

Notation

Usually we use another notation for writing the substitution $\sigma := \{x \mapsto a, y \mapsto f(b), z \mapsto a\}$ which is $[a/x], [f(b)/y], [a/z]$ where $[A/X]$ (read as A *for* X i.e. substitute X with A) $\equiv X \mapsto A \equiv (X, A) \in \sigma$

Def 12.30. Let σ be a substitution, A a formula, and A' an alphabetical variant of A, such that $\text{intro}(\sigma) \cap \text{BVar}(A') = \emptyset$. Then we define **capture-avoiding substitution application** via $\sigma(A) := \sigma(A')$

Note

We said $\sigma(\forall X.A) = \forall X.\sigma_{-X}(A)$ but this can lead to **variable capture**, i.e. $[f(X)/Y](\forall X.p(X,Y))$ would evaluate to $\forall X.p(X,f(X))$ where the second occurrence of X is now bound whereas it was free before.
The solution for this is to rename the bound variable X in $\forall X.p(X,Y)$ before applying the substitution and there where we need capture-avoiding substitution application

Example 12.5

we want to apply $\sigma := \{[f(X)/Y]\}$ to the formula $A = \forall X.p(X,Y)$

- $\text{intro}(\sigma) \cap \text{BVar}(A) = \{X\} \cap \{X\} = \{X\}$
- Hence, we need to find an alphabetical variant A' for A, say $A' = \forall Z.p(Z,Y)$
- Now $\text{intro}(\sigma) \cap \text{BVar}(A') = \{X\} \cap \{Z\} = \emptyset$
- Hence, we define $\sigma(A) = \sigma(A')$
- $\sigma(A') = \sigma(\forall Z.p(Z,Y)) = \forall Z.\sigma_{-Z}(p(Z,Y))$
- $[f(X)/Y](p(Z,Y)) = p(Z,f(X))$
- $\sigma(A) = \sigma(A') = \forall Z.p(Z,f(X))$

Def 12.31. Unification: For given terms A_1, A_2, **unification** is the problem of finding a substitution σ (called **unifier**), such that $\sigma(A_1) = \sigma(A_2)$

Def 12.32. Unification Problem: We call a formula $A^1 =^? B^1 \wedge \cdots \wedge A^n =^? B^n$ a **unification problem** iff $A^i, B^i \in \text{wff}_\iota(\Sigma_1, V_\iota)$

Def 12.33. A substitution is called a **unifier** for a unification problem $\mathcal{E}$ (and thus $\mathcal{E}$ is **unifiable**), iff it is a (simultaneous) unifier for all pairs in $\mathcal{E}$

Def 12.34. The solution for a unification problem $U(A_1, A_2)$ is the set of unifiers $\{\sigma \mid \sigma(A_1) = \sigma(A_2)\}$. For example, let $\mathcal{E} := f(X) =^? f(g(Y))$, solutions are, e.g.

- $[g(a)/X], [a/Y]$
- $[g(g(a))/X], [g(a)/Y]$
- $[g(Z)/X], [Z/Y]$
- $[g(Y)/X]$

Def 12.35. Let σ and θ be substitutions and $W \subseteq \mathcal{V}_\iota$, we say that a substitution σ is **more general** than θ on W (write as $\sigma \leq \theta[W]$), iff there is a substitution ρ, such that $\theta = \rho \circ \sigma[W]$, where $\sigma = \rho[W]$, iff $\sigma(X) = \rho(X)$ for all $X \in W$

Note

- The substitution $\sigma = [g(Z)/X, Z/Y]$ is more general than $\theta = [g(a)/X, a/Y]$ because there exists $\rho = [a/Z]$ such that:

  $$\theta = \rho \circ \sigma$$

  This identity $\theta = \rho \circ \sigma$ is verified by applying the composition to the variables in the domain:
  - $(\rho \circ \sigma)(X) = \rho(\sigma(X)) = \rho(g(Z)) = g(a)$
  - $(\rho \circ \sigma)(Y) = \rho(\sigma(Y)) = \rho(Z) = a$

  Since the mapping for each variable matches θ, we confirm that θ is a specialization of σ via ρ.
- Likewise, σ is more general than $\theta' = [g(g(a))/X, g(a)/Y]$ via $\rho' = [g(a)/Z]$, since:

  $$\theta' = \rho' \circ \sigma$$

  where $(\rho' \circ \sigma)(X) = g(g(a))$ and $(\rho' \circ \sigma)(Y) = g(a)$.

Def 12.36. MGU: σ is called a **most general unifier (mgu)** of A_1, A_2, iff it is minimal in $U(A_1, A_2)$ **with respect to** the more general ordering ($\leq$) restricted to the variables occurring in the problem: $W = \text{free}(A_1) \cup \text{free}(A_2)$.

Def 12.37. We call a pair $A =^? B$ **solved** in a unification problem $\mathcal{E}$, iff A is a variable X, $\mathcal{E} = X =^? A \wedge \mathcal{E}'$, and $X \notin (\text{free}(A) \cup \text{free}(\mathcal{E}'))$. We call a unification problem $\mathcal{E}$ a **solved form**, iff all its pairs are **solved**.

Lemma 12.1

Solved forms are of the form $X^1 =^? B^1 \wedge \cdots \wedge X^n =^? B^n$ where X^i are distinct and $X^i \notin \text{free}(B^j)$

Def 12.38. $\mathcal{E}_\sigma$**:** Any substitution $\sigma = [B^1/X^1], \cdots, [B^n/X^n]$ induces a **solved unification problem**

$$\mathcal{E}_\sigma := (X^1 =^? B^1 \wedge \cdots \wedge X^n =^? B^n)$$

Lemma 12.2

If $\mathcal{E} = X^1 =^? B^1 \wedge \cdots \wedge X^n =^? B^n$ is a solved form, then $\mathcal{E}$ has the unique MGU $\sigma_\epsilon := [B^1/X^1], \cdots, [B^n/X^n]$.

Def 12.39. $\mathcal{U}$**:** A unification engine $\mathcal{U}$ uses the following inference rules:

$$\frac{\mathcal{E} \wedge f(A^1, \cdots, A^n) =^? f(B^1, \cdots, B^n)}{\mathcal{E} \wedge A^1 =^? B^1 \wedge \cdots \wedge A^n =^? B^n}\ \mathcal{U}_{dec} \qquad \frac{\mathcal{E} \wedge A =^? A}{\mathcal{E}}\ \mathcal{U}_{triv}$$

$$\frac{\mathcal{E} \wedge X =^? A \quad X \notin free(A) \quad X \in free(\mathcal{E})}{[A/X](\mathcal{E} \wedge X =^? A)}\ \mathcal{U}_{elim} \qquad \frac{\mathcal{E} \wedge f(X^1, \cdots, X^n) =^? g(Y^1, \cdots, Y^n)}{\bot}\ \mathcal{U}\bot$$

Example 12.6: Unification

Let $\mathcal{E} := f(g(X, X), h(a)) =^? f(g(a, Z), h(Z))$:

$$\frac{\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{f(g(X,X),h(a)) =^? f(g(a,Z),h(Z))}{g(X,X) =^? g(a,Z) \wedge h(a) =^? h(Z)}\ \mathcal{U}_{dec}}{X =^? a \wedge X =^? Z \wedge h(a) =^? h(Z)}\ \mathcal{U}_{dec}}{X =^? a \wedge X =^? Z \wedge a =^? Z}\ \mathcal{U}_{elim}}{X =^? a \wedge a =^? Z \wedge a =^? Z}\ \mathcal{U}_{elim}}{X =^? a \wedge Z =^? a \wedge a =^? a}\ \mathcal{U}_{triv}}{X =^? a \wedge Z =^? a}$$

The final $\mathcal{E}$ is in solved form since all the pairs are solved. Hence $\mathcal{E}$ has the unique MGU $\sigma_\epsilon := [a/X], [a/Z]$.

Example 12.7: Unification

Let $\mathcal{E} := S_1(g(f(x), g(x, y)), y) =^? S_1(g(z, v), f(w))$

$$\frac{\dfrac{\dfrac{\dfrac{\dfrac{S_1(g(f(x),g(x,y)),y) =^? S_1(g(z,v),f(w))}{g(f(x),g(x,y)) =^? g(z,v) \wedge y =^? f(w)}\ \mathcal{U}_{dec}}{z =^? f(x) \wedge v =^? g(x,y) \wedge y =^? f(w)}\ \mathcal{U}_{dec}}{z =^? f(x) \wedge v =^? g(x,y) \wedge y =^? f(w)}\ \mathcal{U}_{elim}}{z =^? f(x) \wedge v =^? g(x, f(w)) \wedge y =^? f(w)}\ \mathcal{U}_{elim}$$

The final equation $z =^? f(x) \wedge v =^? g(x, f(w)) \wedge y =^? f(w)$ is in solved form.
The MGU is $\{z \to f(x), v \to g(x, f(w)), y \to f(w)\}$.

Example 12.8: Unification Failure

Let $\mathcal{E} := S_2(g(f(x), g(x,u)), f(y), z) =^? S_2(g(g(g(u,v), f(w)), f(c)), f(g(u,v)), f(c))$

$$\dfrac{\dfrac{\dfrac{S_2(g(f(x), g(x,u)), f(y), z) =^? S_2(g(g(g(u,v), f(w)), f(c)), f(g(u,v)), f(c))}{g(f(x), g(x,u)) =^? g(g(g(u,v), f(w)), f(c)) \wedge f(y) =^? f(g(u,v)) \wedge z =^? f(c)}\,\mathcal{U}_{dec}}{f(x) =^? g(g(u,v), f(w)) \wedge g(x,u) =^? f(c) \wedge f(y) =^? f(g(u,v)) \wedge z =^? f(c)}\,\mathcal{U}_{dec}}{\bot}\,\mathcal{U}\bot$$

12.3 The Resolution Calculus

Def 12.40. Test Calculus: For a given conjecture A and hypotheses $\mathcal{H}$, a **test calculus** $\mathcal{T}$ tries to derive a refutation $\mathcal{H}, \neg A \vdash_{\mathcal{T}} \bot$ instead of $\mathcal{H} \vdash A$, where $\neg A$ is unsatisfiable iff A is valid and $\bot$ an unsatisfiable proposition.

Def 12.41. A formula is called **atomic** (or an **atom**) if it does not contain logical constants, otherwise, **complex**

Def 12.42. Labeled Formula: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, $A \in \mathcal{L}$ a formula, L a **label set**, and $\alpha \in L$ a **label**, then we call a pair $\langle A, \alpha \rangle$ a **labeled formula** and write it as A^α. For a set Φ of propositions we use $\Phi^\alpha := \{A^\alpha \mid A \in \Phi\}$

Note

If the label set is $\mathcal{D}_0 := \{T, F\}$, we call a labeled formula A^T **positive** and A^F **negative**

Note

Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, and A^α a labeled formula. Then we say that $M \in \mathcal{M}$ **satisfies** A^α (write $M \models A^\alpha$), iff $\alpha = T$ and $M \models A$ or $\alpha = F$ and $M \not\models A$

Def 12.43. Literal: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, $A \in \mathcal{L}$ is an atomic formula, and $\alpha \in \{T, F\}$, then we call the labeled formula A^α a **literal**.

Def 12.44. Opposite Literal: For a literal A^α, we call the literal A^β with $\alpha \neq \beta$ the opposite literal.

Def 12.45. CNF: A formula is in **conjunctive normal form (CNF)** if it is $\top$ or a conjunction of disjunctions of literals, i.e. if it is of the form:

$$\bigwedge_{i=1}^{n} \bigvee_{j=1}^{m_i} l_{ij} := \underbrace{\overbrace{(l_{1,1} \vee \cdots \vee l_{1,m_1})}^{\text{Clause } i=1 \text{ has } m_1 \text{ literals}} \wedge \overbrace{(l_{2,1} \vee \cdots \vee l_{2,m_2})}^{\text{Clause } i=2 \text{ has } m_2 \text{ literals}} \wedge \cdots \wedge \overbrace{(l_{n,1} \vee \cdots \vee l_{n,m_n})}^{\text{Clause } i=n \text{ has } m_n \text{ literals}}}_{\text{There are } n \text{ total clauses}}$$

Def 12.46. The **resolution calculus** (or simply **resolution**) is a test calculus consisting of the inference rules shown in Figure 2

$$\dfrac{P^T \vee A \qquad P^F \vee B}{A \vee B}\,\mathcal{R}_0$$

$$\dfrac{A^T \vee C \quad B^F \vee D \quad \sigma = \text{MGU}(A,B)}{(\sigma(C)) \vee (\sigma(D))}\,\mathcal{R}_1 \qquad \dfrac{A^\alpha \vee B^\beta \vee C \quad \sigma = \text{MGU}(A,B)}{(\sigma(A)) \vee (\sigma(C))}\,\mathcal{R}_1$$

Figure 2: Resolution Calculus

Resolution is the main machinery behind the Prolog inference engine

Def 12.47. Clause: A **clause** is a disjunction $l_1^{\alpha_1} \vee \cdots \vee l_n^{\alpha_n}$ of literals. We use $\square$ for the *"empty"* disjunction (no disjuncts) and call it the **empty clause**. A clause with exactly one literal is called a **unit clause**.

Def 12.48. Clause Set: We often write a **clause set** $\{C_1, \cdots, C_n\}$ as $C_1; \cdots; C_n$, and write $S; T$ for the union of the

clause sets S and T, and $S;C$ for the extension of S by a clause C.

Def 12.49. Resolution Refutation: Let S be a clause set, then we call an $\mathcal{R}$-derivation of the empty clause $\square$ from S **$\mathcal{R}$-refutation** and write $\mathcal{D} : S \vdash_{\mathcal{R}} \square$.

Example 12.9: Resolution Proof

We want to prove the following propositional formula using resolution:

$$(p \Rightarrow q \Rightarrow r) \Rightarrow (p \Rightarrow q) \Rightarrow p \Rightarrow r$$

We negate it and try to derive the empty clause (refutation). After negating it and transforming it to CNF (since resolution works only on CNF), we get the following clause set:

$$\Phi := \{p^F \vee q^F \vee r^T, p^F \vee q^T, p^T, r^F\}$$

We start resolving:

1	$p^F \vee q^F \vee r^T$	initial
2	$p^F \vee q^T$	initial
3	p^T	initial
4	r^F	initial
5	$p^F \vee q^F$	resolve 1.3 with 4.1
6	q^F	resolve 3.1 with 5.1
7	p^F	resolve 2.2 with 6.1
8	$\square$	resolve 3.1 with 7.1

Note

A resolution proof can be represented in a proof tree as shown in Figure 3

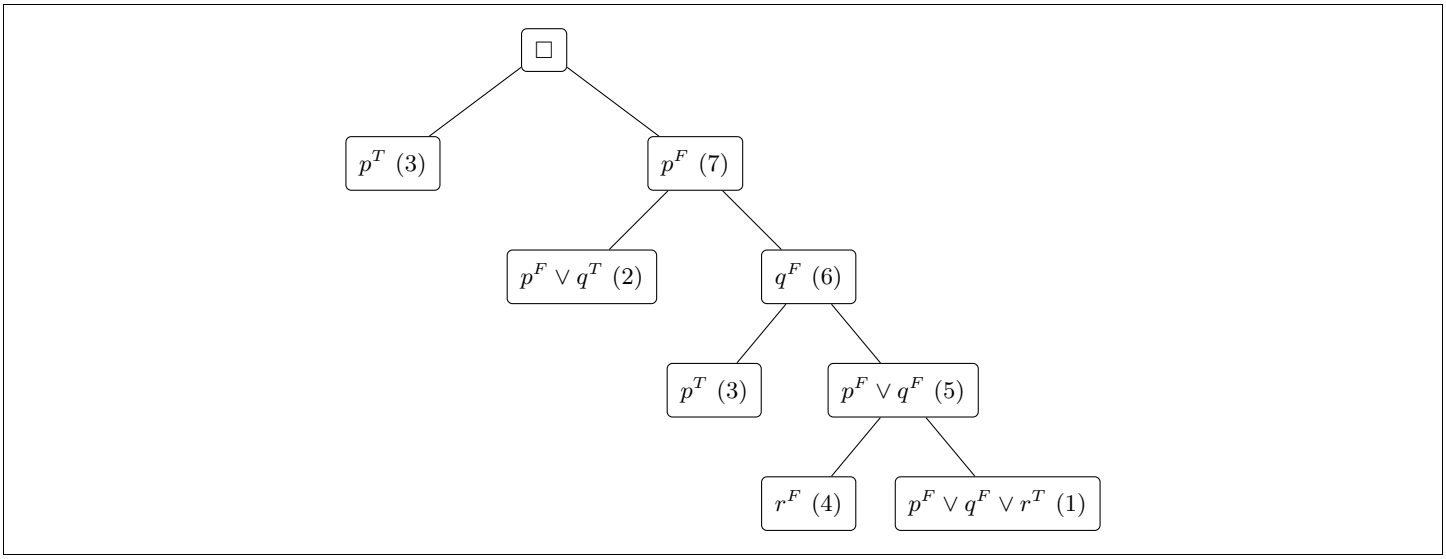


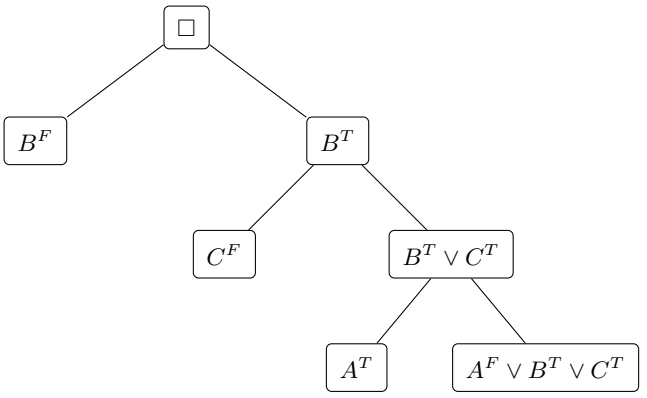
Figure 3: Resolution Proof Tree

Example 12.10: Resolution Given a Knowledge Base

Consider we have a knowledge base KB $:= \{A \Rightarrow B \vee C, A, \neg B\}$, we want to check if KB $\models C$. KB $\models C$ iff KB $\cup \{\neg C\}$ is unsatisfiable.
First we need to compute CNF$(A \Rightarrow B \vee C)$

- CNF$(A \Rightarrow B \vee C) = \neg A \vee B \vee C$

KB $\cup \{\neg C\} := \{\neg A \vee B \vee C, A, \neg B, \neg C\} := \{A^F \vee B^T \vee C^T, A^T, B^F, C^F\}$



Since $\text{KB} \cup \{\neg C\}$ is unsatisfiable, $\text{KB} \vDash C$

Def 12.50. Refutational Completeness: A test calculus $\mathcal{C}$ is called **refutationally complete** iff for every unsatisfiable clause set Φ, there exists a refutation deriving the empty clause:

$$\Phi \text{ is unsatisfiable} \iff \Phi \vdash_{\mathcal{C}} \square$$

Deductive vs. Refutational Completeness

Resolution is **not** deductively complete: from the clause set $\{p\}$, resolution cannot directly derive the valid consequence $p \vee q$. However, resolution is **sound and refutationally complete**: if we add the negation $\neg(p \vee q) \equiv p^F \wedge q^F$ to $\{p^T\}$, resolution is guaranteed to derive the contradiction $\square$.

12.4 Horn Logic and SLD-Resolution

Def 12.51. A **Horn clause** is a clause with **at most** one **positive** literal

Observation

- A rule in Prolog $H :- B_1, \ldots, B_n$ corresponds to $H^T \vee B_1^F \vee \cdots \vee B_n^F$, i.e. a Horn clause
- A goal set (query) $?- G_1, \ldots, G_n$ asks for $G_1 \wedge \cdots \wedge G_n$, which by refutation negates to the clause $G_1^F \vee \cdots \vee G_n^F$
- A fact F corresponds to the unit clause F^T

Def 12.52. Horn Logic: Let $S := \langle \mathcal{L}, \mathcal{M}, \vDash \rangle$ be a logical system. A formal system $\langle S, \mathcal{C} \rangle$ is called a **Horn Logic** if:

- The language $\mathcal{L}$ is the set of Horn clauses.
- The calculus $\mathcal{H}$ is defined by the rules of Modus Ponens (MP), Conjunction Introduction ($\wedge I$), and Substitution (`Subst`).

Def 12.53. SLD-Resolution: **SLD-Resolution** (**S**elected literal, **L**inear resolution for **D**efinite clauses) is the formal inference calculus that executes Prolog programs.

Conceptually, Prolog treats a query $\leftarrow A_1, A_2, \ldots, A_k$ (in Prolog syntax: `?- A1, A2, ..., Ak`) as a **list of subgoals** that must all be satisfied:

At each execution step:

1. **Selection:** Prolog picks the leftmost subgoal A_1 from the list.
2. **Clause Lookup & Unification:** Prolog searches the program for a clause $H \leftarrow B_1, \ldots, B_m$ (whose variables are renamed to avoid name clashes) such that A_1 and the clause head H unify with MGU $\sigma = \text{MGU}(A_1, H)$.
3. **Goal Reduction (Replacement):** The subgoal A_1 is discharged and **replaced** by the requirements $B_1, \ldots, B_m$ from the clause body. The substitution σ is applied across the entire new goal list:

$$\leftarrow \sigma(B_1, \ldots, B_m,\ A_2, \ldots, A_k)$$

(If H is a simple fact with no body, A_1 is removed without introducing new subgoals).

This reduction repeats until the goal list is completely empty ($\leftarrow \square$), which is called an **SLD-refutation**. The composition of all unifying substitutions σ collected along the derivation forms the **answer substitution**.

The Name Behind the Acronym

- **S (Selected literal):** Instead of searching randomly, Prolog uses a fixed selection rule: always pick the **first (leftmost)** subgoal.
- **L (Linear):** Every step is linear: it always resolves the **current query** against an input clause from the program; it never wastes time resolving two rules against each other.
- **D (Definite clauses):** The program is restricted to definite Horn clauses (each clause has exactly one head literal).

Def 12.54. Answer Substitution: Let P be a logic program (set of Horn clauses) and Q a query. A substitution σ is called an **answer substitution** for Q if:

- **Proof-theoretically:** There is a derivation D of Q from P, where σ is the composition of all `Subst` instances in D.
- **Operationally:** There is an SLD-refutation of $P \cup \{\leftarrow Q\}$ with sequence of MGUs $\theta_1, \ldots, \theta_n$, such that $\sigma = (\theta_1 \circ \cdots \circ \theta_n)$ restricted to the variables of Q.

Example 12.11: Answer Substitution

Consider the following program:

```
human(leibniz).
human(socrates).
greek(socrates).
fallible(X):-human(X).
```

Where we query `?-fallible(X), greek(X).`
We have three facts, one rule, and one query. They translate to the following clause set:

1. $\text{human}(\text{leibniz})^T$
2. $\text{human}(\text{socrates})^T$
3. $\text{greek}(\text{socrates})^T$
4. $\text{fallible}(X)^T \vee \text{human}(X)^F$ $(\text{human}(X) \Rightarrow \text{fallible}(X))$
5. $\text{fallible}(X)^F \vee \text{greek}(X)^F$ (Start Resolution)
6. Resolve (1) with (4) under $[\text{leibniz}/X] \rightsquigarrow \text{fallible}(\text{leibniz})^T$
7. Resolve (5) with (6) under $[\text{leibniz}/X] \rightsquigarrow \text{greek}(\text{leibniz})^F$
8. $\text{greek}(\text{leibniz})^F$ cannot be resolved further; the program backtracks to try another substitution.
9. Resolve (2) with (4) under $[\text{socrates}/X] \rightsquigarrow \text{fallible}(\text{socrates})^T$
10. Resolve (5) with (9) under $[\text{socrates}/X] \rightsquigarrow \text{greek}(\text{socrates})^F$
11. Resolve (3) with (10) $\rightsquigarrow \{\square\}$
12. The program returns T and the answer substitution $\sigma := [\text{socrates}/X]$.

Note

Prolog's query engine natively executes backchaining (SLD-resolution). However, from a proof-theoretic standpoint, the exact same Horn knowledge base can also establish truths via forward chaining!

Forward Chaining

A bottom-up, data-driven derivation that starts with known facts and repeatedly applies program rules (via Modus Ponens and substitution) to infer new facts until the desired conclusion is produced.

Example 12.12: Backchaining

Consider the program:

```
has_motor(c).
has_wheels(c,4).
car(X) :- has_motor(X), has_wheels(X,4).
```

And the query `?-car(X).`
We have two facts, one rule, and one query. They translate to the following clause set:

1. $\mathrm{m}(c)^T$
2. $\mathrm{w}(c,4)^T$
3. $\mathrm{c}(X)^T \vee \mathrm{m}(X)^F \vee \mathrm{w}(X,4)^F$ $(\mathrm{m}(X) \wedge \mathrm{w}(X,4) \Rightarrow \mathrm{c}(X))$
4. $\mathrm{c}(X)^F$ (Start Resolution)
5. Resolve (3) with (4) $\rightsquigarrow \mathrm{m}(X)^F \vee \mathrm{w}(X,4)^F$
6. Resolve (5) with (1) under $[c/X] \rightsquigarrow \mathrm{w}(c,4)^F$
7. Resolve (6) with (2) $\rightsquigarrow \{\square\}$
8. The program returns T and the answer substitution $\sigma := [c/X]$.

Example 12.13: Forward Chaining in Horn Logic

Consider the same program. Instead of Prolog's top-down query execution, we examine how the formal calculus of Horn Logic $\mathcal{H}$ derives truths **forward** (bottom-up) from the program's clauses taken as axioms:

1. $m(c)$
2. $w(c,4)$
3. $m(X) \wedge w(X,4) \Rightarrow \mathrm{car}(X)$

Using the inference rules of Horn logic, the logical consequence $\mathrm{car}(c)$ is derived as follows:

$$\dfrac{\dfrac{\dfrac{}{m(c)}\,Ax \quad \dfrac{}{w(c,4)}\,Ax}{m(c) \wedge w(c,4)}\,\wedge I \quad \dfrac{\dfrac{}{m(X) \wedge w(X,4) \Rightarrow car(X)}\,Ax}{m(c) \wedge w(c,4) \Rightarrow car(c)}\,\text{Subst } [c/X]}{\mathrm{car}(c)}\,MP$$

Conclusion

- **Forward Chaining:** A bottom-up derivation in $\mathcal{H}$ starts from the axioms and builds up to the conclusion $\mathrm{car}(c)$.
- **Backward Chaining:** Prolog starts with the query $\leftarrow \mathrm{car}(X)$ and uses SLD-refutation to reduce goals top-down to $\square$, computing the exact same answer substitution $\sigma = [c/X]$.

References

[NM90] Ulf Nilsson and Jan Maluszynski. *Logic, programming and Prolog*. John Wiley & Sons, Inc., 1990.

[Fla94] Peter Flach. "Simply logical: intelligent reasoning by example". In: (1994).

[SS94] Leon Sterling and Ehud Y Shapiro. *The art of Prolog: advanced programming techniques*. MIT press, 1994.

[Brn01] Paul Brna. *Prolog programming: A first course*. 2001.

[End07] Ulle Endriss. "An introduction to Prolog programming". In: *Institution for logic language and computation* (2007).

[Koh26] Michael Kohlhase. "Artificial Intelligence 1 Winter Semester 2024/25–Lecture Notes". In: (2026).